

GNR 640 - Geospatial Statistics
MINI PROJECT

Submitted by
ANAGHA MURALI
25D0319

Data shared at figshare: <https://doi.org/10.6084/m9.figshare.32147500>

TABLE OF CONTENTS

1. PROJECT OVERVIEW	1
1.1. OBJECTIVES	1
2. DATA SOURCES	2
2.1. STATION OBSERVATIONS (USCRN)	2
2.2. REFERENCE DATA - (ERA5 - LAND)	3
3. METHODOLOGY	4
3.1. DATA PREPARATION	4
3.2. SPATIAL INTERPOLATION	4
3.2.1. INVERSE DISTANCE WEIGHTING (IWD)	4
3.2.2. ORDINARY KRIGING	4
3.3. ERA5 - LAND PROCESSING	5
3.4. PERFORMANCE EVALUATION	6
3.4.1. METRIC - ROOT MEAN SQUARE ERROR (RMSE)	6
3.4.1. METRIC - PEARSON CORRELATION COEFFICIENT (r)	6
3.4. DISTRIBUTION SIMILARITY- KS TWO-SAMPLE TEST	6
3.5. SEASONALITY ANALYSIS	7
4. RESULTS	8
4.1. METRICS - RMSE AND CORRELATION VALUES	8
4.1.1. SAMPLE SCATTEROGRAMS AND BIAS MAPS	10
4.1.2. KEY FINDINGS	13
4.2. STATISTICAL ANALYSIS	14
4.2.1. CENTRAL TENDENCY AND DISPERSION	14
4.2.2. BOX PLOTS	16
4.2.3. KEY FINDINGS	17

4.3. DISTRIBUTIONAL SIMILARITY - KS TWO-SAMPLE TEST	18
4.3.1. SUMMARY OF THE KS TEST VALUES FOR THE VARIOUS VARIABLES	18
4.3.2. KEY FINDINGS	20
4.4. SEASONALITY ANALYSIS	21
4.4.1. AIR TEMPERATURE	21
4.4.2. PRECIPITATION	27
4.4.3. RELATIVE HUMIDITY	30
4.4.4. SOIL TEMPERATURE	34
4.4.5. SOIL MOISTURE	37
5. MODEL SUITABILITY AND RECOMMENDATIONS	41
6. APPENDIX	44

LIST OF FIGURES

Figure 1: USCRN Stations	2
Figure 2: Different Variogram Models	5
Figure 3: RMSE Value Comparison (Different Units)	8
Figure 4: Pearson Correlation r Value Comparison	9
Figure 5: Air Temperature (Interpolated and ERA)	10
Figure 6: Precipitation (Interpolated and ERA)	10
Figure 7: Relative Humidity (Interpolated and ERA)	11
Figure 8: Difference or Bias Maps for Data of 15-07-2010 (Air Temperature)	12
Figure 9: Difference or Bias Maps for Data of 15-07-2010 (Precipitation)	13
Figure 10: Air Temperature Boxplot (2006-2021)	16
Figure 11: Soil Temperature Boxplot (2006-2021)	17
Figure 12: KS Test D-Statistic Heatmap	20
Figure 13: Seasonal Air Mean temperatures	22
Figure 14: Air Temperature (IDW) for dates (15-01-2010,15-04-2010,15-07-2010,15-10-2010)	23
Figure 15: Air Temperature Comparison by different methods for 15-07-2010	24

LIST OF FIGURES

Table 1: Hydroclimatic variables	2
Table 2: Area Extent and Data	4
Table 3: Tested Variogram Models	5
Table 4: D Values and Similarity Values	6
Table 5: Seasons and Months	5
Table 6: RMSE values	8
Table 7: Pearson Correlation (r) values	9
Table 8: Central Tendency and Dispersion Values - Air Temperature	14
Table 9: Central Tendency and Dispersion Values - Precipitation	14
Table 10: Central Tendency and Dispersion Values - Relative Humidity	15
Table 11: Central Tendency and Dispersion Values - Soil Temperature	15
Table 12: Central Tendency and Dispersion Values - Soil Moisture	16
Table 13: KS Values and Similarity Interpretation for Air Temperature, Precipitation and Relative Humidity	18
Table 14: KS Values and Similarity Interpretation for Soil Temperature and Soil Moisture	19
Table 15: Seasonality Analysis of Air Temperature (KS D values)	21
Table 16: Winter season central tendency and dispersion parameters for Air Temperature	24

Table 17: Spring season central tendency and dispersion parameters for Air Temperature	25
Table 18: Summer season central tendency and dispersion parameters for Air Temperature	26
Table 19: Autumn season central tendency and dispersion parameters for Air Temperature	26
Table 20: Seasonality Analysis of Precipitation (KS D values)	27
Table 21: Winter season central tendency and dispersion parameters for Precipitation	28
Table 22: Spring season central tendency and dispersion parameters for Precipitation	28
Table 23: Summer season central tendency and dispersion parameters for Precipitation	29
Table 24: Autumn season central tendency and dispersion parameters for Precipitation	30
Table 25: Seasonality Analysis of Relative Humidity (KS D values)	30
Table 26: Winter season central tendency and dispersion parameters for Relative Humidity	31
Table 27: Spring season central tendency and dispersion parameters for Relative Humidity	32
Table 28: Summer season central tendency and dispersion parameters for Relative Humidity	32
Table 29: Autumn season central tendency and dispersion parameters for Relative Humidity	33
Table 30: Seasonality Analysis of Soil Temperature (KS D values)	34
Table 31: Winter season central tendency and dispersion parameters for Soil Temperature	34

Table 32: Spring season central tendency and dispersion parameters for Soil Temperature	35
Table 33: Summer season central tendency and dispersion parameters for Soil Temperature	36
Table 34: Autumn season central tendency and dispersion parameters for Soil Temperature	36
Table 35: Seasonality Analysis of Soil Moisture (KS D values)	37
Table 36: Winter season central tendency and dispersion parameters for Soil Moisture	38
Table 37: Spring season central tendency and dispersion parameters for Soil Moisture	38
Table 38: Summer season central tendency and dispersion parameters for Soil Moisture	39
Table 39: Autumn season central tendency and dispersion parameters for Soil Moisture	40
Table 40: Best Model for the various Variables	41

Data shared at figshare: <https://doi.org/10.6084/m9.figshare.32147500>

1. PROJECT OVERVIEW

This project focuses on generating and evaluating gridded hydroclimatic data products for the contiguous United States using station-based observations. The work involves geospatial interpolation of point measurements from the United States Climate Reference Network (USCRN) to produce gridded datasets at 2-degree spatial resolution and daily temporal resolution, followed by statistical evaluation against ERA5-Land reanalysis data from the European Centre for Medium-Range Weather Forecasts (ECMWF).

1.1. OBJECTIVES

1. Generate gridded data products at 2-degree spatial and daily temporal resolution for five hydroclimatic variables using station-based USCRN observations.
2. Compare generated gridded datasets against ERA5-Land using RMSE and Pearson correlation coefficient.
3. Perform statistical analysis including central tendency (mean, median), dispersion (variance, standard deviation), and distributional similarity using the Kolmogorov-Smirnov two-sample test.
4. Analyse seasonal patterns of each variable across the four meteorological seasons.
5. Recommend the most suitable interpolation model based on accuracy metrics.

2. DATA SOURCES

2.1. STATION OBSERVATIONS (USCRN)

Station observations were obtained from the United States Climate Reference Network (USCRN), managed by NOAA. The network comprises over 130 stations distributed across the contiguous United States, Alaska, and Hawaii. The dataset provides pre-processed daily aggregated values for the following five hydroclimatic variables:

Table 1: Hydroclimatic variables

Variable	Unit	Description
Air Temperature	°C	Daily mean 2m air temperature
Precipitation	mm	Daily accumulated precipitation total
Relative Humidity	%	Daily mean relative humidity
Soil Temperature	°C	Daily mean soil temperature at 10cm depth
Soil Moisture	m ³ /m ³ (fraction)	Daily mean volumetric soil water content

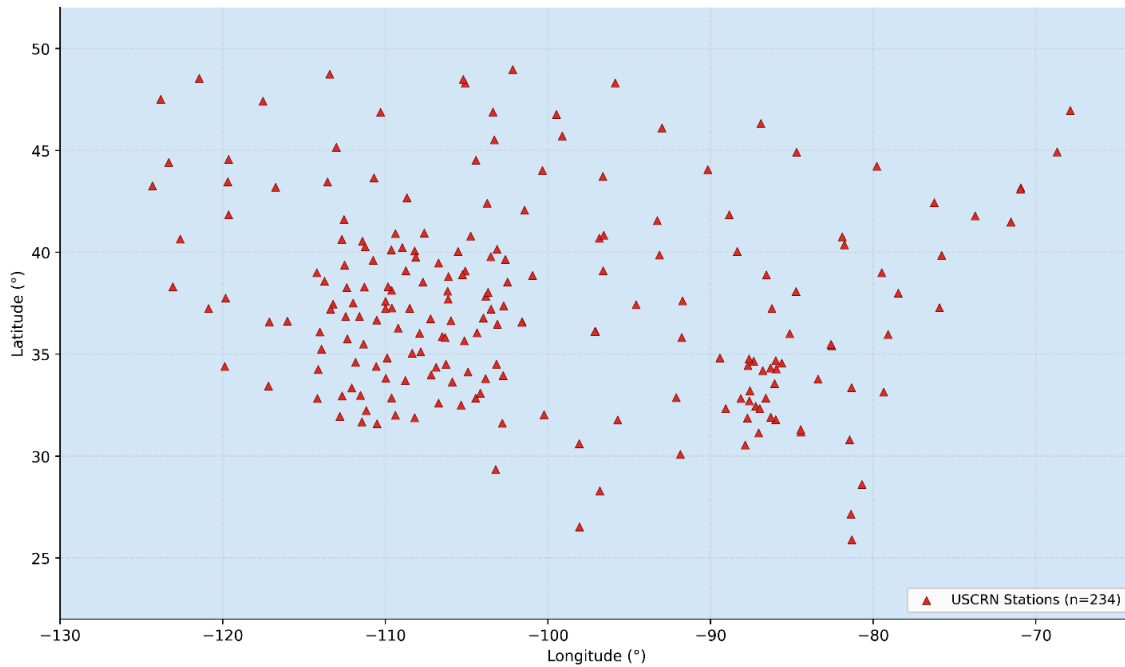


Figure 1: USCRN Stations

2.2. REFERENCE DATA - (ERA5 - LAND)

RA5-Land reanalysis data was used as the independent reference dataset for evaluating the generated gridded products. ERA5-Land is produced by ECMWF by replaying the land component of the ERA5 climate reanalysis. Key characteristics include:

- Global spatial coverage at 0.1-degree (~9km) resolution.
- Daily temporal resolution (aggregated from hourly model output).
- Historical records from 1950 to present.
- Variables used: 2m temperature (t2m), 2m dewpoint (d2m), soil temperature layer1 (stl1), volumetric soil water layer 1 (swvl1), total precipitation (tp).
- ERA5-Land was downloaded from the Copernicus Climate Data Store (CDS) using the cdsapi Python library.
- Daily aggregated values were extracted for each variable >> daily means for temperature, dewpoint, soil temperature and soil moisture.
- Daily summation for precipitation.

3. METHODOLOGY

3.1. DATA PREPARATION

Station observations were loaded from pre-processed Excel files containing daily time series for each variable at each USCRN station. Station metadata (latitude, longitude, elevation) was merged with the time series data. A 2-degree resolution grid was defined covering the contiguous USA:

Table 2: Area Extent and Data

Parameter	Value
Longitude range	-124° to -68° (West to East)
Latitude range	25° to 49° (South to North)
Grid spacing	2 degrees
Grid dimensions	13 latitude points × 29 longitude points = 377 grid cells
Total grid - days	5844 days × 377 cells = 2203188 values per variable

3.2. SPATIAL INTERPOLATION

Two interpolation methods were applied to generate gridded daily fields from the station point observations for all five variables across all 5844 days:

3.2.1. INVERSE DISTANCE WEIGHTING (IWD)

IDW assigns weights to surrounding station observations inversely proportional to their distance from the target grid point. IDW is computationally efficient and performs well for smoothly varying fields such as temperature.

3.2.2. ORDINARY KRIGING

Ordinary Kriging is a geostatistical interpolation method that models spatial autocorrelation through a variogram. Three variogram models were tested:

Table 3: Tested Variogram Models

Variogram Model	Characteristics
Spherical	Most stable, Recommended for sparse networks
Exponential	Reaches sill asymptotically, Good for gradual variation
Gaussian	Very smooth, Can be unstable with sparse data

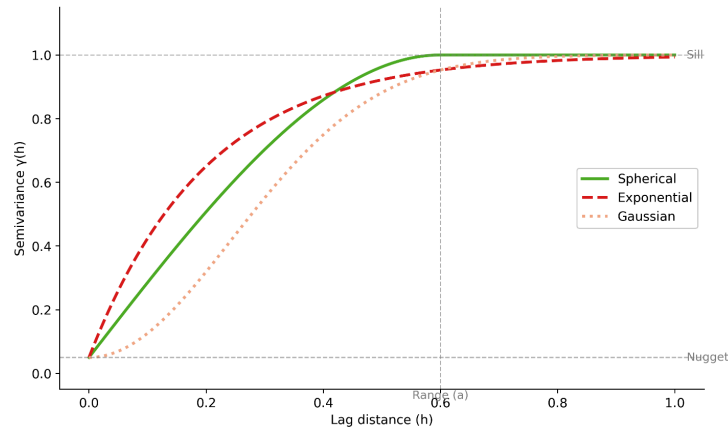


Figure 2: Different Variogram Models

3.3. ERA5 - LAND PROCESSING

- Downloaded ERA5-Land at 0.1-degree resolution for the USA bounding box (24°N - 50°N, 125°W - 67°W).
- Daily aggregated values were extracted for each variable >> daily means for temperature, soil variables and dewpoint.
- Daily summation for precipitation and converted to appropriate units (Kelvin to °C for temperature; metres to mm for precipitation).
- Relative humidity was computed from ERA5 2m temperature (t2m) and 2m dewpoint temperature (d2m).
- ERA5 was spatially coarsened from 0.1° to 2° resolution using spatial averaging over 20 × 20 grid blocks.
- The coarsened ERA5 grid was aligned to the interpolation grid using linear interpolation to ensure exact spatial correspondence between ERA5 and interpolated grids.

3.4. PERFORMANCE EVALUATION

Two metrics were used to quantify agreement between interpolated and ERA5 gridded fields.

3.4.1. METRIC - ROOT MEAN SQUARE ERROR (RMSE)

RMSE measures the average magnitude of errors in the same units as the variable. Lower RMSE indicates better agreement with ERA5.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (P_i - R_i)^2}$$

Where P is the predicted value , R is the reference value , i is the index of the specific data point and n is the total number of observations or data points

3.4.1. METRIC - PEARSON CORRELATION COEFFICIENT (r)

r measures the linear association between interpolated and ERA5 values. Values range from -1 to 1, with 1 indicating perfect positive correlation.

$$r = \frac{cov(P,R)}{((\sigma_p)(\sigma_R))}$$

Where $cov(P,R)$ is the covariance between predicted and real data points, σ_p is the standard deviation of the predicted value and σ_R is the standard deviation of the reference value

3.4. DISTRIBUTION SIMILARITY- KS TWO-SAMPLE TEST

The Kolmogorov-Smirnov (KS) two-sample test was used to assess whether the interpolated and ERA5 values follow the same statistical distribution. The KS test statistic D represents the maximum absolute difference between the two cumulative distribution functions (CDFs).

Table 4: D Values and Similarity Values

D - Value	Similarity
$D \leq 0.05$	Very similar
$0.05 < D \leq 0.15$	Slight difference (Slight diff)

$0.15 < D \leq 0.30$	Moderate difference (Moderate diff)
$D > 0.30$	Large difference (Large diff)

3.5. SEASONALITY ANALYSIS

Seasonal analysis was performed by grouping daily interpolated values into the four standard meteorological seasons and computing mean values across all grid points for each season. The seasons are defined as:

Table 5: Seasons and Months

Season	Abbreviation	Months
Winter	DJF	December, January, February
Spring	MAM	March, April, May
Summer	JJA	June, July, August
Autumn	SON	September, October, November

4. RESULTS

4.1. METRICS - RMSE AND CORRELATION VALUES

Performance metrics were computed across all grid points and all 5844 days for each variable and interpolation method

Table 6: RMSE values

Variable	IDW	Kriging Spherical	Kriging Exponential	Kriging Gaussian
Air Temperature (°C)	3.2023	2.8568	3.2737	4.6263
Precipitation (mm)	4.2132	4.2347	4.5166	3.7159
Relative Humidity (%)	11.167	10.939	11.017	12.880
Soil Temperature (°C)	4.4810	4.4113	4.5096	5.0081
Soil Moisture (fraction)	0.0957	0.0939	0.0967	0.0978

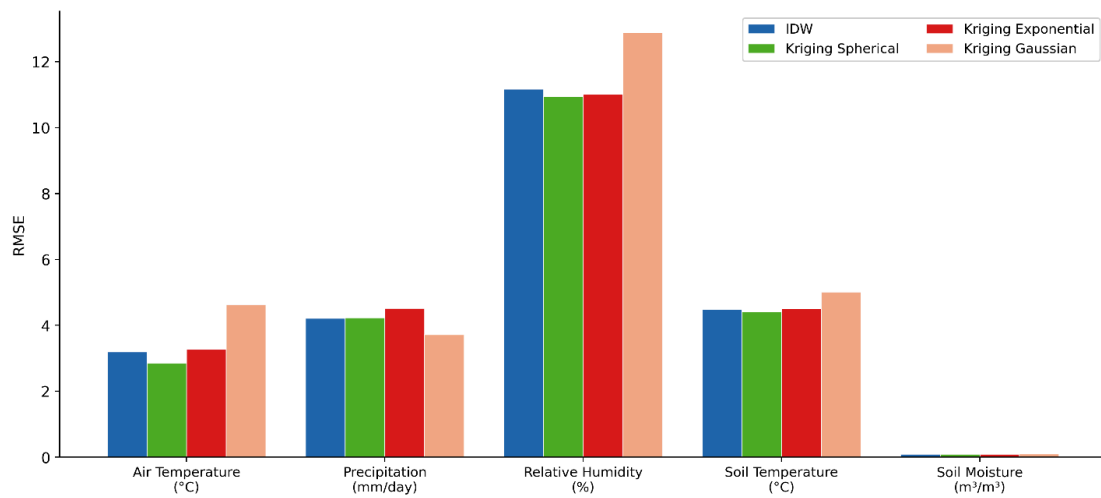


Figure 3: RMSE Value Comparison (Different Units)

Table 7: Pearson Correlation (r) values

Variable	IDW	Kriging Spherical	Kriging Exponential	Kriging Gaussian
Air Temperature	0.9624	0.9699	0.9602	0.9248
Precipitation	0.6478	0.6293	0.4417	0.6501
Relative Humidity	0.8318	0.8405	0.8391	0.7912
Soil Temperature	0.9296	0.9379	0.9319	0.8964
Soil Moisture	0.6607	0.6835	0.6502	0.6323

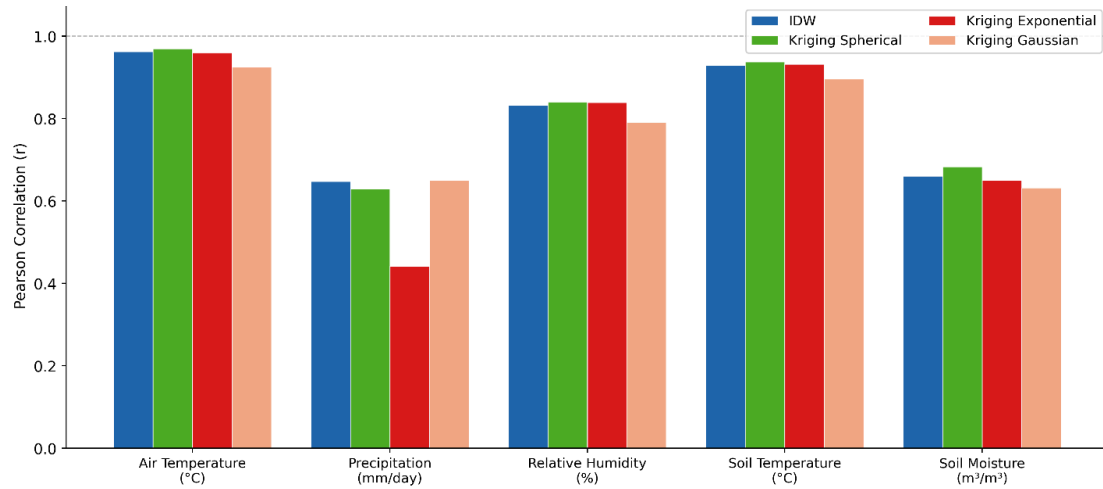


Figure 4: Pearson Correlation r Value Comparison

4.1.1. SAMPLE SCATTEROGRAMS AND BIAS MAPS

Some of the example scatterograms

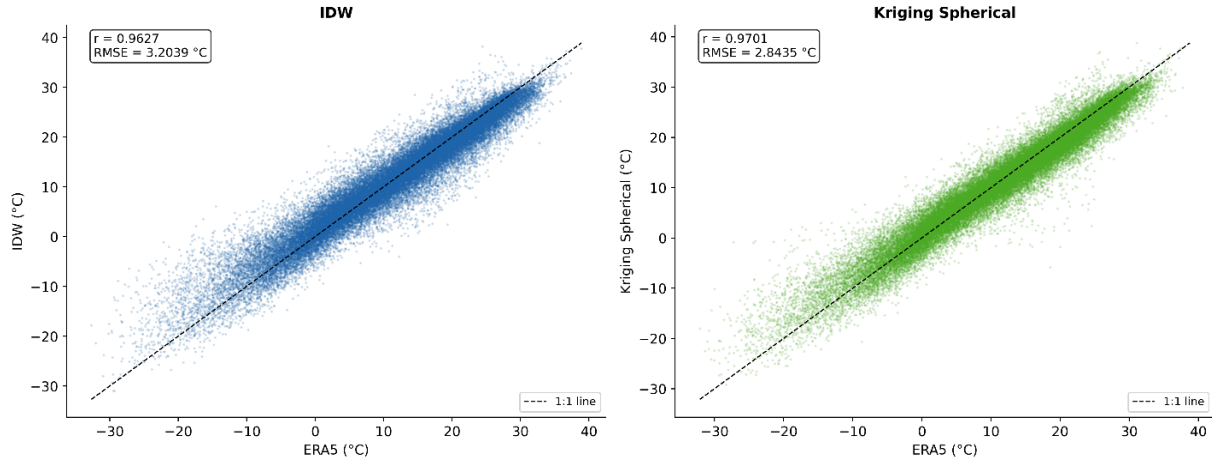


Figure 5: Air Temperature (Interpolated and ERA)

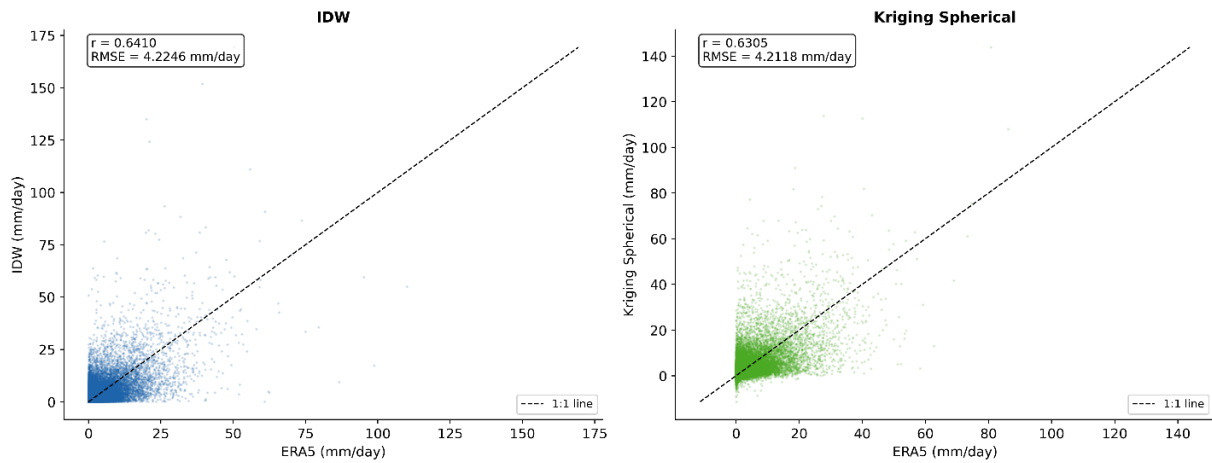


Figure 6: Precipitation (Interpolated and ERA)

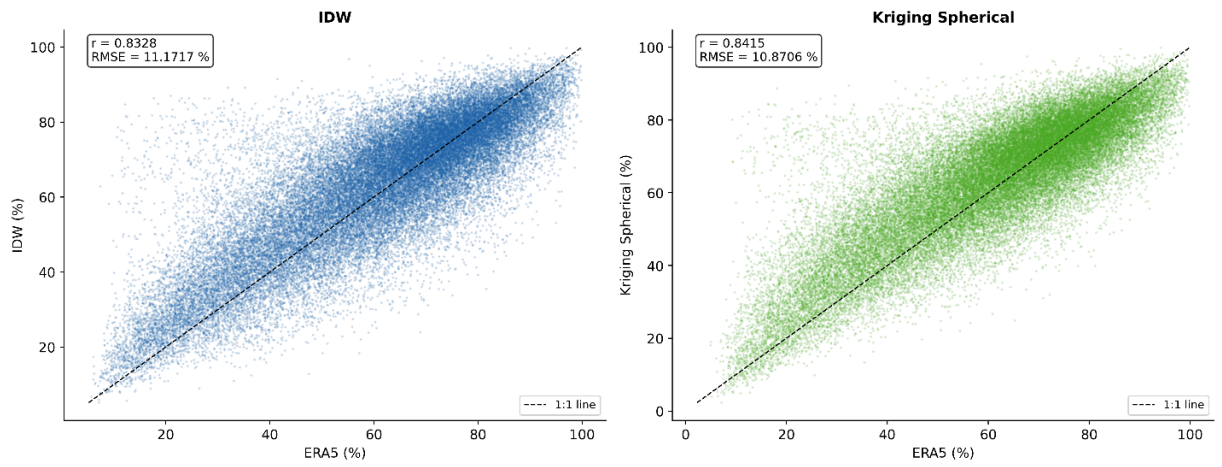


Figure 7: Relative Humidity (Interpolated and ERA)

Difference or Bias Maps for few variables

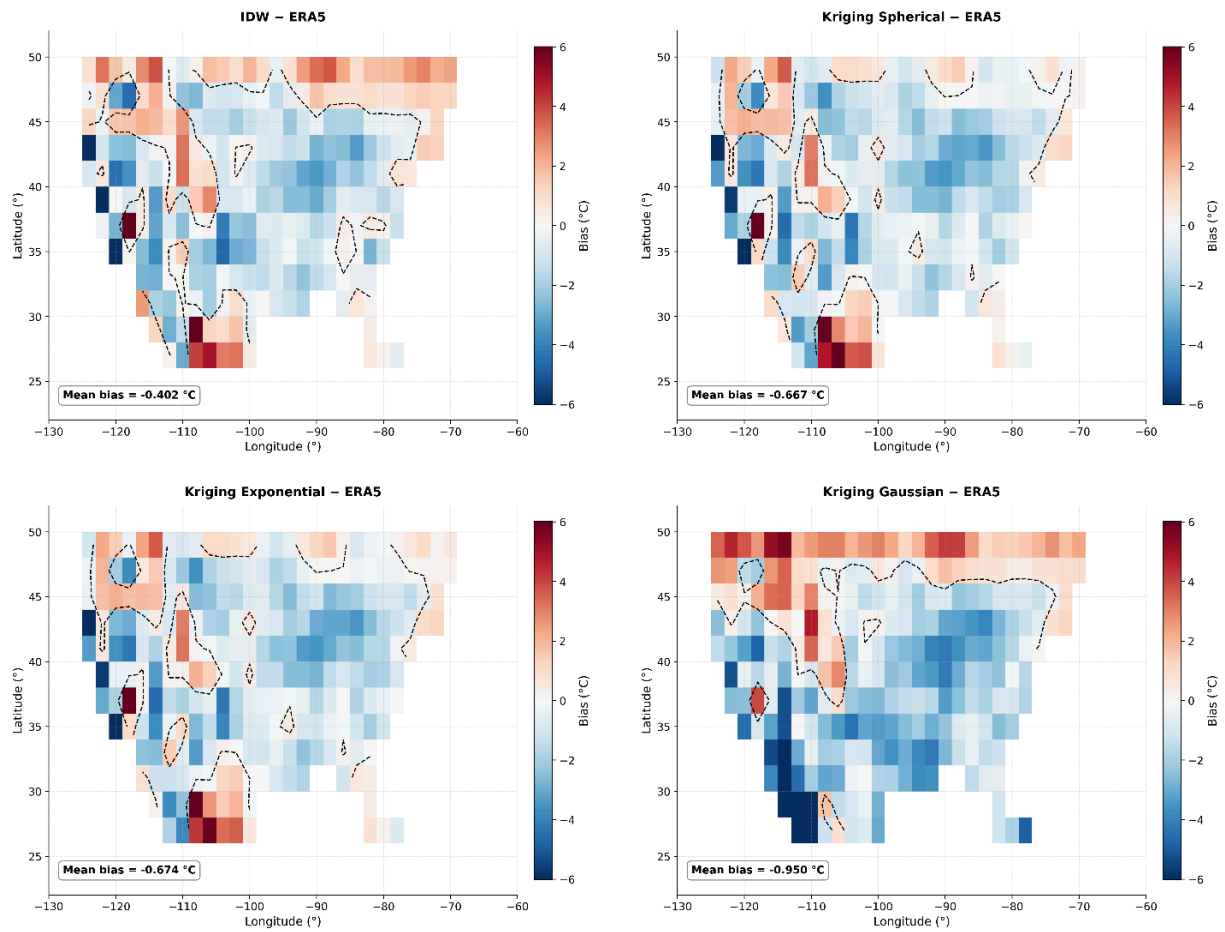


Figure 8: Difference or Bias Maps for Data of 15-07-2010 (Air Temperature)

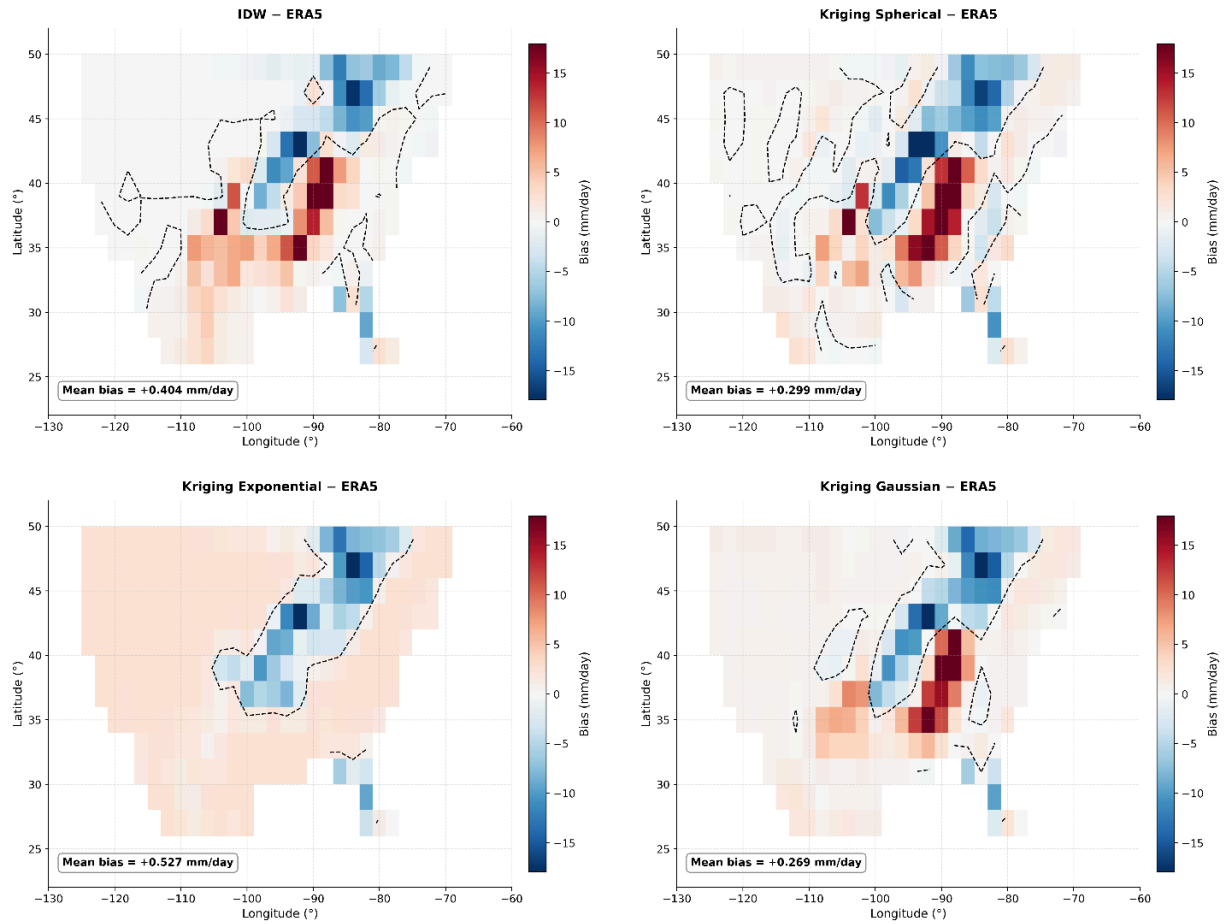


Figure 9: Difference or Bias Maps for Data of 15-07-2010 (Precipitation)

4.1.2. KEY FINDINGS

- Air temperature achieved the highest correlation ($r = 0.97$ for Kriging spherical) with low RMSE of 2.86°C >> excellent interpolation performance.
- Soil temperature also performed well ($r = 0.94$, $\text{RMSE} = 4.41^{\circ}\text{C}$) reflecting the smooth spatial variation of soil thermal fields.
- Relative humidity showed good performance ($r = 0.84$, $\text{RMSE} = 10.94\%$), this variable was computed from ERA5 dewpoint data using the Magnus formula.

- Precipitation showed moderate correlation ($r = 0.65$ for IDW), inherently the hardest variable to interpolate due to high spatial variability and convective events.
- Soil moisture showed moderate correlation ($r = 0.68$), limited by the ERA5 swv11 layer depth (0 - 7cm) vs USCRN measurement at 10 cm.
- Kriging spherical consistently performed best or equal best across all variables.
- Gaussian Kriging showed higher RMSE for most variables due to numerical instability with sparse station networks.

4.2. STATISTICAL ANALYSIS

4.2.1. CENTRAL TENDENCY AND DISPERSION

Statistical properties were computed for both IDW interpolated values and ERA5 reference values across all grid points and days.

Table 8: Central Tendency and Dispersion Values - Air Temperature

Method	Mean	Median	Std dev	Variance
IDW	13.0703	14.3126	10.2529	105.1214
Kriging - Spherical	13.2233	14.8264	10.4674	109.5663
Kriging - Exponential	13.0413	14.4332	10.2928	105.9408
Kriging - Gaussian	11.1691	11.5980	9.0251	81.4525
ERA5 (reference)	11.3543	12.6655	11.5999	134.5586

Table 9: Central Tendency and Dispersion Values - Precipitation

Method	Mean	Median	Std dev	Variance
IDW	2.3518	0.3068	5.4188	29.3636

Kriging - Spherical	2.4372	0.9027	5.2905	27.9889
Kriging - Exponential	2.5211	1.7990	3.7329	13.9343
Kriging - Gaussian	2.3763	1.4158	3.6136	13.0579
ERA5 (reference)	2.0650	0.2038	4.7853	22.8986

Table 10: Central Tendency and Dispersion Values - Relative Humidity

Method	Mean	Median	Std dev	Variance
IDW	64.7704	68.5050	17.0053	289.1814
Kriging - Spherical	66.4348	69.7226	16.1402	260.5054
Kriging - Exponential	66.6058	70.2281	16.3836	268.4229
Kriging - Gaussian	66.2070	67.8050	11.1918	125.2573
ERA5 (reference)	60.8995	64.1449	19.5483	382.1371

Table 11: Central Tendency and Dispersion Values - Soil Temperature

Method	Mean	Median	Std dev	Variance
IDW	14.2039	15.5050	11.4802	131.7953
Kriging - Spherical	14.4192	16.1520	11.6921	136.7043
Kriging - Exponential	14.1658	15.6408	11.5368	133.0985
Kriging - Gaussian	12.0812	12.5611	10.2743	105.5614
ERA5 (reference)	13.2406	13.3381	10.5902	112.1519

Table 12: Central Tendency and Dispersion Values - Soil Moisture

Method	Mean	Median	Std dev	Variance
IDW	0.2015	0.1950	0.0869	0.0076
Kriging - Spherical	0.2020	0.1957	0.0714	0.0051
Kriging - Exponential	0.2036	0.1951	0.0747	0.0056
Kriging - Gaussian	0.2083	0.2057	0.0608	0.0037
ERA5 (reference)	0.2546	0.2670	0.1118	0.0125

4.2.2. BOX PLOTS

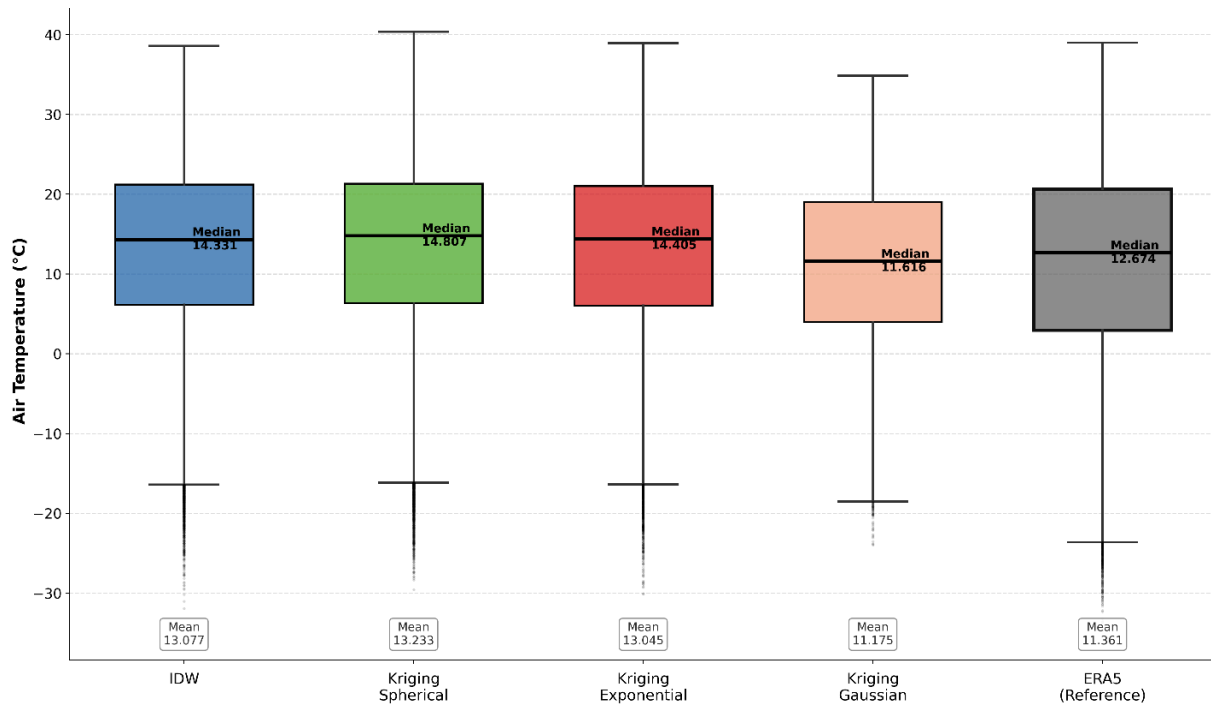


Figure 10: Air Temperature Boxplot (2006-2021)

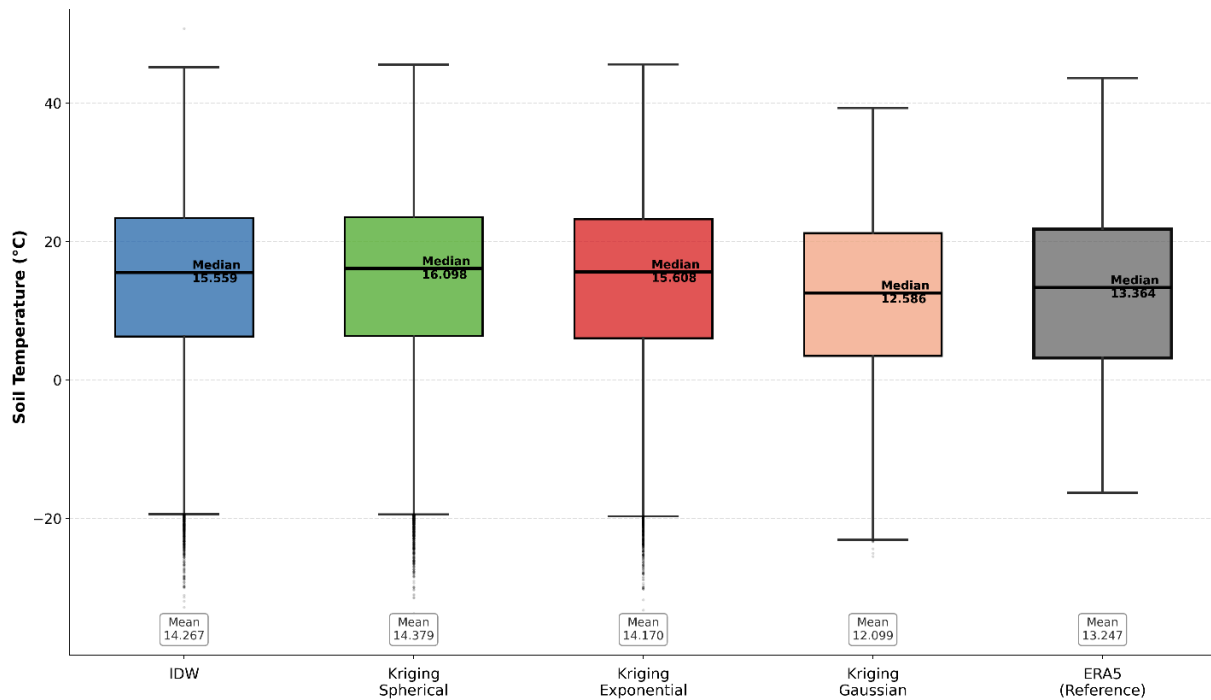


Figure 11: Soil Temperature Boxplot (2006-2021)

4.2.3. KEY FINDINGS

- **Air Temperature** : IDW mean (13.07°C) is very close to ERA5 (11.35°C), with a difference of only +1.72°C. IDW shows a slightly lower variance (105.12 vs 134.56), suggesting the interpolation reduces spatial extremes relative to the reanalysis grid.
- **Precipitation** : IDW mean (2.35 mm/day) shows a slight positive bias (+0.29 mm/day) vs ERA5 (2.07 mm/day) . The ERA5 median (0.20) is lower than IDW (0.31), reflecting ERA5's heavier tail of high-precipitation events captured in variance (22.90 vs 29.36 for IDW).
- **Relative Humidity** : IDW overestimates vs ERA5: IDW mean 64.77% vs ERA5 60.90% is +3.87%. ERA5 shows higher variance (382.14 vs 289.18), indicating ERA5 captures a wider range of dryness extremes that station-based IDW smooths out.

- **Soil Temperature** : IDW mean (14.20°C) is slightly higher than ERA5 (13.24°C) by a value of +0.96°C. IDW median (15.51°C) also exceeds ERA5 median (13.34°C), suggesting IDW skews warmer - likely due to station placement biases toward populated, lower-elevation sites.
- **Soil Moisture** : IDW consistently underestimates vs ERA5: mean 0.2015 vs 0.2546 m³/m³ (difference = -0.0531). ERA5 references the swvl1 layer (0–7 cm), which is inherently wetter than the USCRN 5 cm point measurement, explaining the systematic dry bias in IDW
- The high standard deviation values are expected and physically correct. On any given day across the United States, there is an enormous range of values. When statistics are computed across all grid points covering the entire country for all 16 years, this large within-country spatial contrast drives the standard deviation to appear high. It is not an error but a true reflection of the climatic diversity across the United States.

4.3. DISTRIBUTIONAL SIMILARITY - KS TWO-SAMPLE TEST

4.3.1. SUMMARY OF THE KS TEST VALUES FOR THE VARIOUS VARIABLES

Table 13: KS Values and Similarity Interpretation for Air Temperature, Precipitation and Relative Humidity

Method	Air temp (°C)		Precipitation (mm/day)		Rel. humidity (%)	
	KS D	Similarity	KS D	Similarity	KS D	Similarity
IDW	0.0300	Very similar	0.2184	Moderate diff	0.0506	Slight diff
Kriging - Spherical	0.0334	Very similar	0.2320	Moderate diff	0.0667	Slight diff
Kriging - Exponential	0.0355	Very similar	0.3184	Large diff	0.0642	Slight diff

Table 13: KS Values and Similarity Interpretation for Air Temperature, Precipitation and Relative Humidity

Method	Air temp (°C)		Precipitation (mm/day)		Rel. humidity (%)	
	KS D	Similarity	KS D	Similarity	KS D	Similarity
Kriging - Gaussian	0.0809	Slight diff	0.4413	Large diff	0.1384	Slight diff

Table 14: KS Values and Similarity Interpretation for Soil Temperature and Soil Moisture

Method	Soil temp (°C)		Soil moisture (m ³ /m ³)	
	KS D	Similarity	KS D	Similarity
IDW	0.1020	Slight diff	0.1960	Moderate diff
Kriging - Spherical	0.1150	Slight diff	0.2604	Moderate diff
Kriging - Exponential	0.1092	Slight diff	0.2518	Moderate diff
Kriging - Gaussian	0.0787	Slight diff	0.3059	Large diff

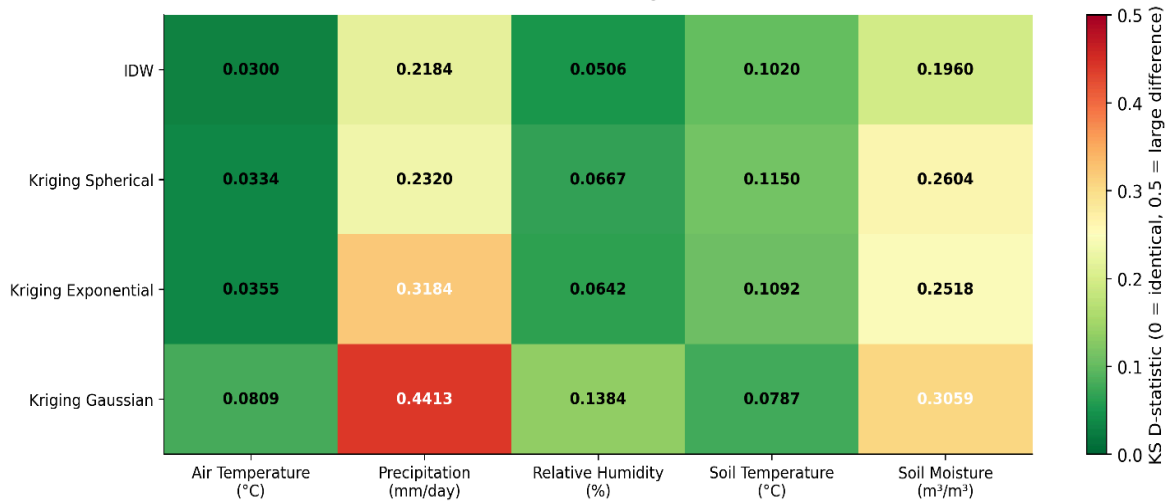


Figure 12: KS Test D-Statistic Heatmap
(Green - Similar Distribution, Red - Large Distributional Difference)

4.3.2. KEY FINDINGS

- **Air Temperature** : All methods show the smallest KS D values overall. IDW ($D = 0.0300$) and Kriging Spherical ($D = 0.0334$) achieve "Very similar" distributions vs ERA5, indicating station-based interpolation faithfully reproduces the temperature frequency distribution.
- **Precipitation** : Largest distributional divergence across all variables. Kriging Gaussian ($D = 0.4413$) and Kriging Exponential ($D = 0.3184$) show "Large diff", while IDW ($D = 0.2184$) performs best - still only "Moderate diff" - reflecting the inherently skewed, zero-inflated nature of precipitation that all spatial interpolation methods struggle to reproduce against ERA5.
- **Relative Humidity** : All four methods consistently fall in "Slight diff" ($D = 0.05 - 0.14$), with IDW ($D = 0.0506$) closest to the boundary of "Very similar". Kriging Gaussian ($D = 0.1384$) diverges most, likely due to its over-smoothing compressing the tails of the humidity distribution.

- **Soil Temperature** : All methods fall in "Slight diff". Notably, Kriging Gaussian ($D = 0.0787$) performs best here - the smoothest kernel matches ERA5's soil temperature distribution better than it does surface variables.
- **Soil Moisture** : Moderate to large distributional differences persist across all methods ($D = 0.196 - 0.306$). Kriging Gaussian ($D = 0.3059$) reaches "Large diff", driven by the systematic dry bias of USCRN 5 cm observations relative to ERA5's swvl1 (0 - 7 cm) layer - a structural mismatch that spatial interpolation cannot correct.

4.4. SEASONALITY ANALYSIS

4.4.1. AIR TEMPERATURE

Table 15: Seasonality Analysis of Air Temperature (KS D values)

Season	IDW	Kriging - Spherical	Kriging - Exponential	Kriging - Gaussian
Winter (DJF)	0.0474 Very similar	0.0300 Very similar	0.0490 Very similar	0.1322 Slight diff
Spring (MAM)	0.0652 Slight diff	0.0434 Very similar	0.0459 Very similar	0.1161 Slight diff
Summer (JJA)	0.0609 Slight diff	0.0674 Slight diff	0.0663 Slight diff	0.1956 Moderate diff
Autumn (SON)	0.0464 Very similar	0.0432 Very similar	0.0503 Slight diff	0.1144 Slight diff

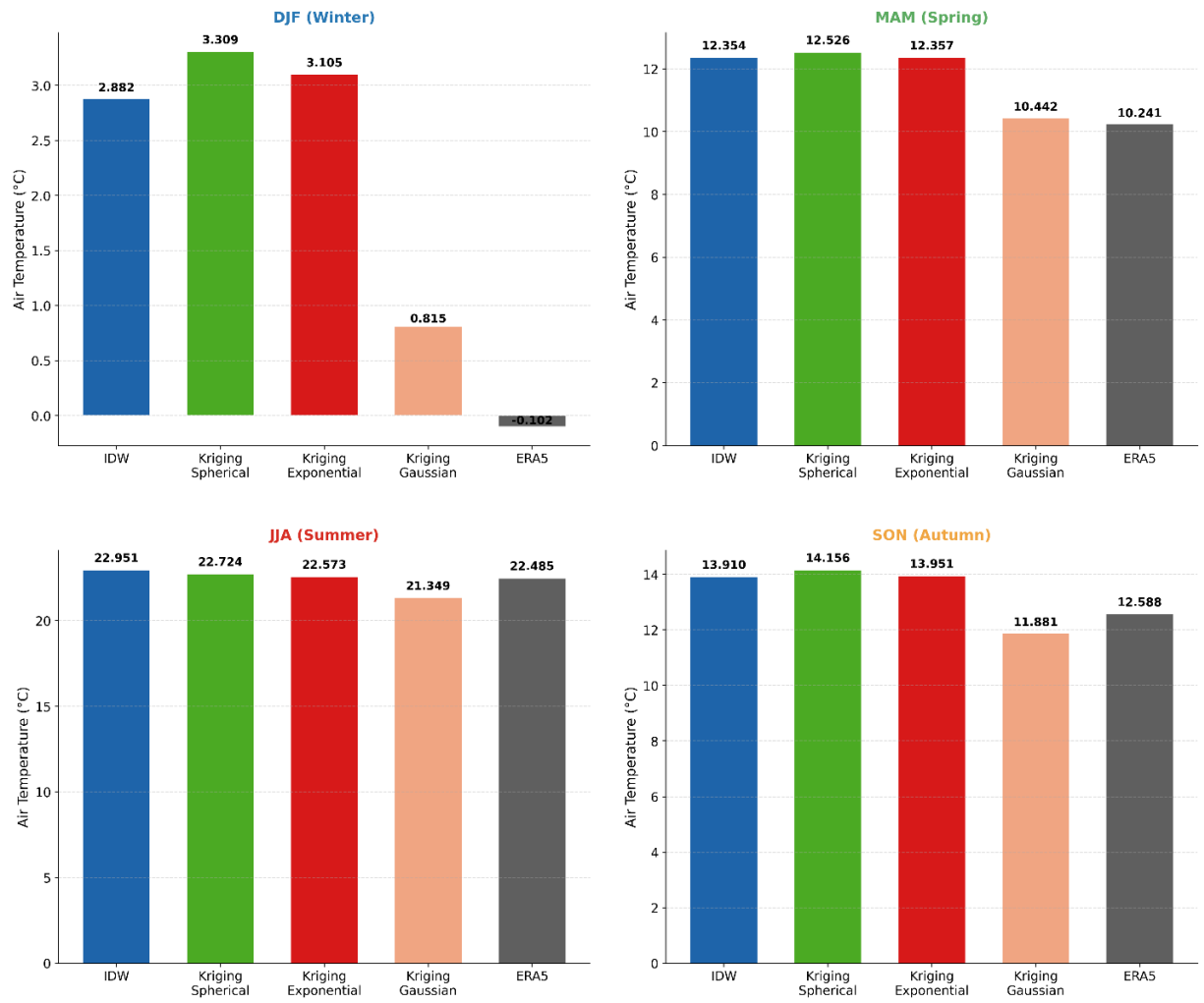


Figure 13: Seasonal Air Mean temperatures

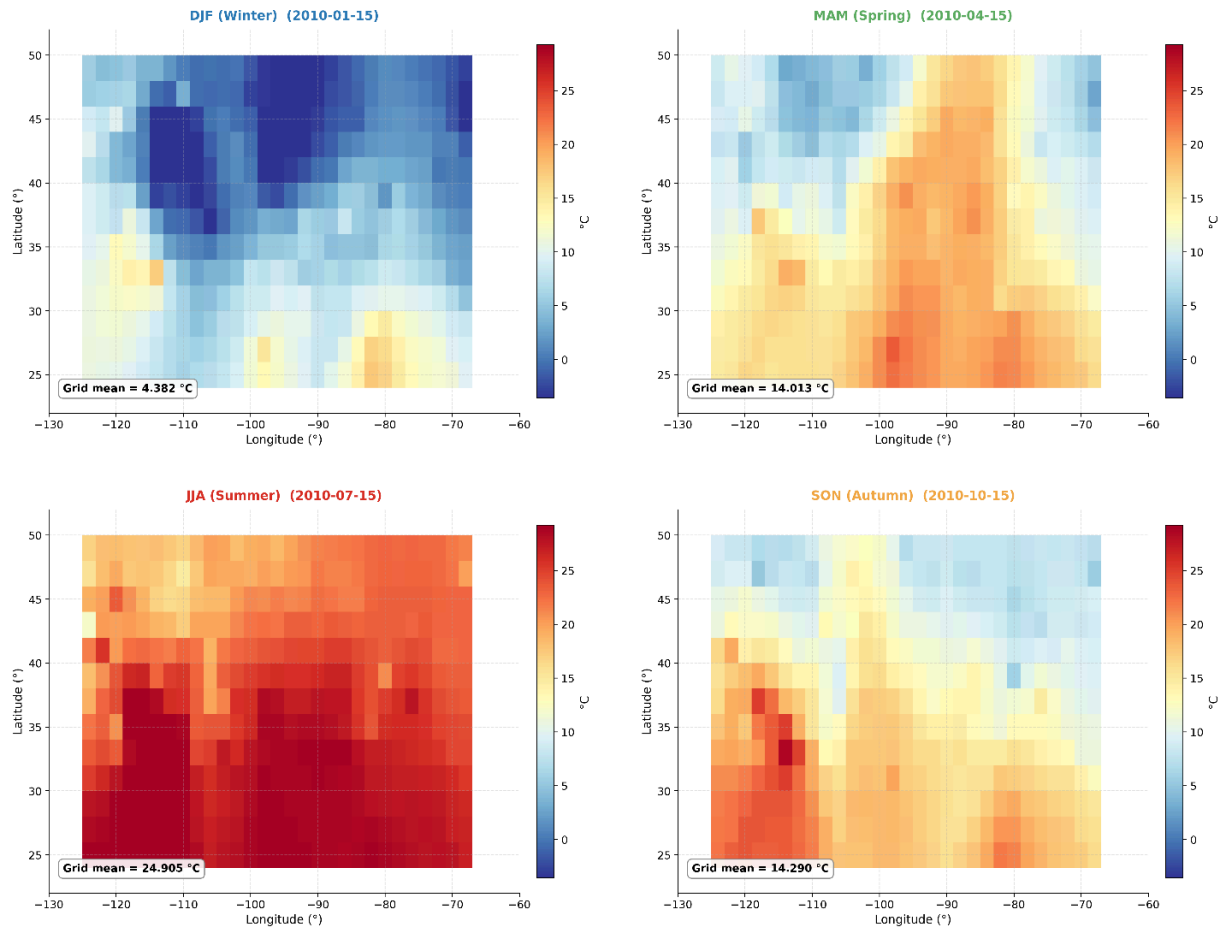


Figure 14: Air Temperature (IDW) for dates (15-01-2010,15-04-2010,15-07-2010,15-10-2010)

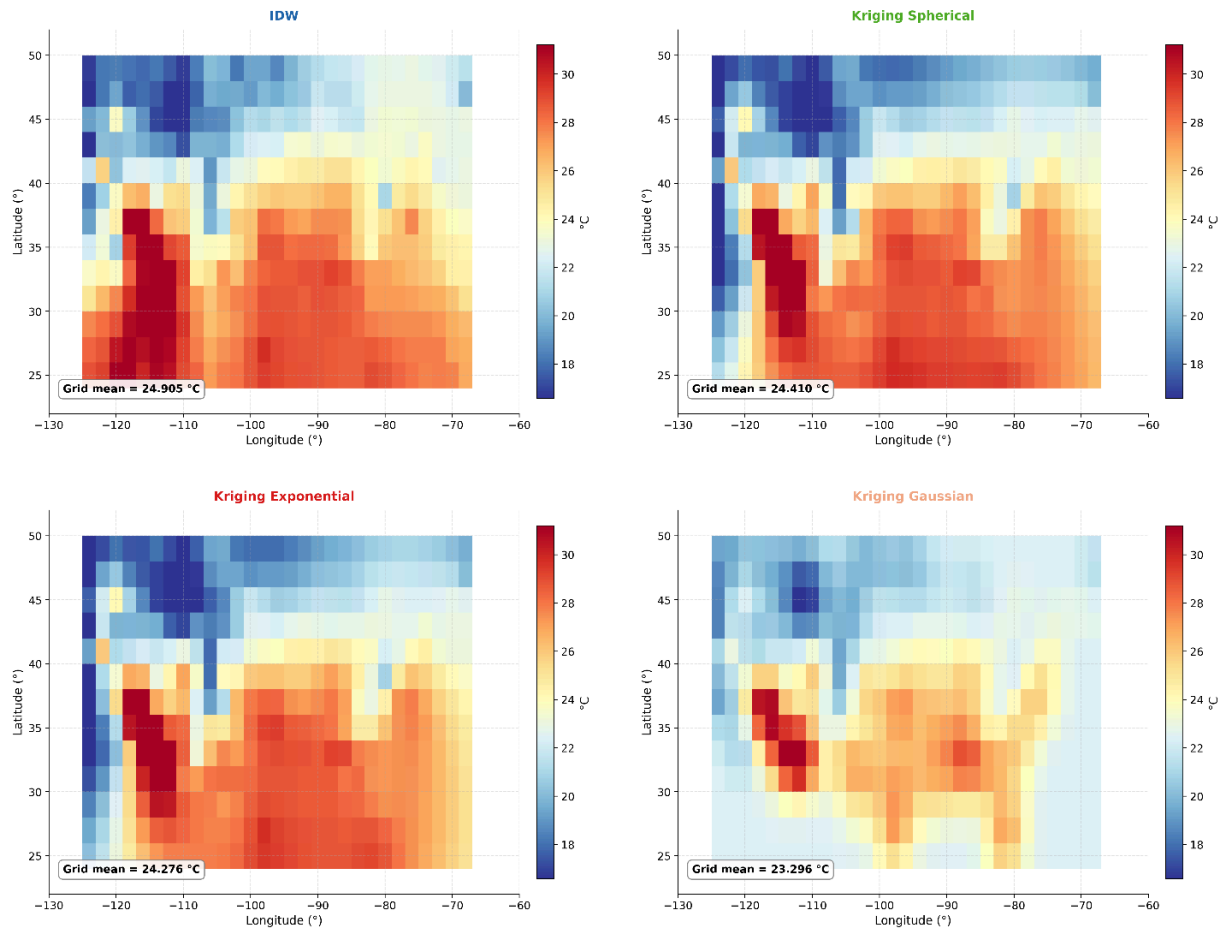


Figure 15: Air Temperature Comparison by different methods for 15-07-2010

Winter (DJF) :

Table 16: Winter season central tendency and dispersion parameters for Air Temperature

Method	Mean	Median	Std dev	Variance
IDW	2.8819	3.4090	8.6680	75.1348
Kriging - Spherical	3.3094	3.6525	9.3873	88.1213
Kriging - Exponential	3.1047	3.1675	8.8933	79.0906

Kriging - Gaussian	0.8148	0.8497	5.3648	28.7808
ERA5 (reference)	-0.1018	-0.1384	9.7373	94.8150

IDW and Kriging Spherical show a warm bias of $\sim 3^{\circ}\text{C}$ relative to ERA5 (-0.10°C) in winter, driven by station placement at lower elevations. Kriging Gaussian severely underperforms with mean of only 0.81°C and collapsed variance (28.78 vs ERA5's 94.82), making it unsuitable for winter temperature interpolation.

Spring (MAM):

Table 17: Spring season central tendency and dispersion parameters for Air Temperature

Method	Mean	Median	Std dev	Variance
IDW	12.3537	12.9601	7.7530	60.1090
Kriging - Spherical	12.5256	13.3211	8.2402	67.9006
Kriging - Exponential	12.3571	13.0476	8.0576	64.9248
Kriging - Gaussian	10.4418	10.6414	5.8779	34.5496
ERA5 (reference)	10.2412	10.4336	9.0385	81.6946

The warm bias reduces to $\sim 2^{\circ}\text{C}$ in spring as the continent warms uniformly. ERA5 variance (81.69) remains higher than all interpolated methods, reflecting ERA5's inclusion of colder high-altitude grid cells that the station network under represents.

Summer (JJA):

Table 18: Summer season central tendency and dispersion parameters for Air Temperature

Method	Mean	Median	Std dev	Variance
IDW	22.9506	23.4771	4.4501	19.8032
Kriging - Spherical	22.7237	23.2145	4.8828	23.8415
Kriging - Exponential	22.5730	23.0853	4.8946	23.9569
Kriging - Gaussian	21.3494	21.2106	3.0203	9.1225
ERA5 (reference)	22.4850	22.8302	5.2960	28.0472

Summer shows the best mean agreement : IDW (22.95°C) is only 0.46°C above ERA5 (22.49°C). The reduced north-south thermal gradient in summer makes this the easiest season to interpolate for air temperature across all methods.

Autumn (SON):

Table 19: Autumn season central tendency and dispersion parameters for Air Temperature

Method	Mean	Median	Std dev	Variance
IDW	13.9103	14.5223	7.9848	63.7565
Kriging - Spherical	14.1559	15.0209	8.3390	69.5383

Kriging - Exponential	13.9513	14.6429	8.1145	65.8457
Kriging - Gaussian	11.8813	12.1557	6.4872	42.0844
ERA5 (reference)	12.5883	13.0261	8.7700	76.9125

Autumn warm bias (IDW 13.91°C vs ERA5 12.59°C) reappears as cooling is faster at high elevations captured by ERA5 but underrepresented in the station network. KS D returns to Very similar for IDW (0.0464) and Kriging Spherical (0.0432).

4.4.2. PRECIPITATION

Table 20: Seasonality Analysis of Precipitation (KS D values)

Season	IDW	Kriging - Spherical	Kriging - Exponential	Kriging - Gaussian
Winter (DJF)	0.2085 Moderate diff	0.2540 Moderate diff	0.2301 Moderate diff	0.4447 Large diff
Spring (MAM)	0.2082 Moderate diff	0.2236 Moderate diff	0.3337 Large diff	0.4248 Large diff
Summer (JJA)	0.1730 Moderate diff	0.1942 Moderate diff	0.4471 Large diff	0.4300 Large diff
Autumn (SON)	0.2843 Moderate diff	0.2784 Moderate diff	0.2869 Moderate diff	0.4861 Large diff

Winter (DJF) :

Table 21: Winter season central tendency and dispersion parameters for Precipitation

Method	Mean	Median	Std dev	Variance
IDW	2.2313	0.2393	5.4495	29.6970
Kriging - Spherical	2.2855	0.6346	5.2618	27.6870
Kriging - Exponential	2.4561	1.1867	4.2421	17.9958
Kriging - Gaussian	2.2705	1.2884	3.6178	13.0888
ERA5 (reference)	1.9081	0.1677	4.6710	21.8179

IDW winter mean (2.23mm) is 0.32mm above ERA5 (1.91mm). Kriging models are poorly suited to precipitation interpolation.

Spring (MAM):

Table 22: Spring season central tendency and dispersion parameters for Precipitation

Method	Mean	Median	Std dev	Variance
IDW	2.3145	0.3024	5.2331	27.3854
Kriging - Spherical	2.3745	0.8737	5.0675	25.6793
Kriging - Exponential	2.4802	1.7943	3.4079	11.6136

Kriging - Gaussian	2.3419	1.3883	3.5278	12.4455
ERA5 (reference)	2.2083	0.2267	4.8422	23.4472

Spring shows the best mean agreement - IDW (2.31mm) vs ERA5 (2.21mm).

Summer (JJA):

Table 23: Summer season central tendency and dispersion parameters for Precipitation

Method	Mean	Median	Std dev	Variance
IDW	2.6027	0.6093	5.1671	26.6991
Kriging - Spherical	2.6715	1.3946	5.0131	25.1317
Kriging - Exponential	2.6997	2.3102	2.9879	8.9275
Kriging - Gaussian	2.5304	1.6415	3.4528	11.9220
ERA5 (reference)	2.1078	0.3240	4.4628	19.9168

IDW summer mean (2.60mm) is 0.49mm above ERA5 (2.11mm) as IDW spreads convective rainfall across broader areas. IDW achieves the lowest seasonal D-statistic for precipitation (0.1730 - Moderate difference), making summer the best season for distributional agreement.

Autumn (SON):

Table 24: Autumn season central tendency and dispersion parameters for Precipitation

Method	Mean	Median	Std dev	Variance
IDW	2.2551	0.1635	5.7998	33.6380
Kriging - Spherical	2.4144	0.6842	5.7818	33.4292
Kriging - Exponential	2.4463	1.5345	4.1558	17.2708
Kriging - Gaussian	2.3600	1.3507	3.8417	14.7583
ERA5 (reference)	2.0326	0.1077	5.1364	26.3823

Autumn has the worst distributional match - IDW $D=0.2843$ and Kriging Gaussian $D=0.4861$ (Large difference), the worst result across all variables and seasons. The transition between convective and frontal precipitation regimes creates mixed spatial structures that all methods struggle to capture

4.4.3. RELATIVE HUMIDITY

Table 25: Seasonality Analysis of Relative Humidity (KS D values)

Season	IDW	Kriging - Spherical	Kriging - Exponential	Kriging - Gaussian
Winter (DJF)	0.0295 Very similar	0.0559 Slight diff	0.0411 Very similar	0.1062 Slight diff

Spring (MAM)	0.0315 Very similar	0.0532 Slight diff	0.0439 Very similar	0.1168 Slight diff
Summer (JJA)	0.1146 Slight diff	0.1226 Slight diff	0.1260 Slight diff	0.1883 Moderate diff
Autumn (SON)	0.0782 Slight diff	0.0902 Slight diff	0.0935 Slight diff	0.1494 Slight diff

Winter (DJF) :

Table 26: Winter season central tendency and dispersion parameters for Relative Humidity

Method	Mean	Median	Std dev	Variance
IDW	68.1103	70.4437	15.2462	232.4461
Kriging - Spherical	68.9000	70.8604	13.5928	184.7638
Kriging - Exponential	69.3159	71.6192	14.1867	201.2615
Kriging - Gaussian	69.8701	71.0302	9.9545	99.0921
ERA5 (reference)	67.2465	70.6629	17.3693	301.6927

Winter shows the closest mean agreement : IDW (68.11%) vs ERA5 (67.25%), a difference of only 0.86%. KS D for IDW (0.0295 - Very similar) is the best seasonal result for relative humidity across all methods

Spring (MAM):

Table 27: Spring season central tendency and dispersion parameters for Relative Humidity

Method	Mean	Median	Std dev	Variance
IDW	61.2025	63.6277	17.1916	295.5528
Kriging - Spherical	62.6518	64.7151	15.5636	242.2265
Kriging - Exponential	62.9060	65.2622	16.0310	256.9919
Kriging - Gaussian	62.7157	63.8032	11.1336	123.9561
ERA5 (reference)	58.6422	61.4122	19.6664	386.7662

IDW spring mean (61.20%) is 2.56% above ERA5 (58.64%). Despite this offset, KS D for IDW (0.0315 - Very similar) confirms the distribution shape is excellently reproduced even when the mean differs.

Summer (JJA):

Table 28: Summer season central tendency and dispersion parameters for Relative Humidity

Method	Mean	Median	Std dev	Variance
IDW	63.9832	69.6311	17.7842	316.2780
Kriging - Spherical	66.5223	71.8962	18.0469	325.6901

Kriging - Exponential	66.3398	72.1580	17.8346	318.0719
Kriging - Gaussian	64.9448	67.1733	11.3252	128.2591
ERA5 (reference)	56.4950	60.0924	20.4189	416.9335

Summer shows the largest bias - IDW (63.98%) is 7.48% above ERA5 (56.50%). ERA5 produces much drier conditions through its land surface evapotranspiration model, a systematic offset that cannot be corrected through interpolation method choice.

Autumn (SON):

Table 29: Autumn season central tendency and dispersion parameters for Relative Humidity

Method	Mean	Median	Std dev	Variance
IDW	65.9075	69.9563	16.8560	284.1245
Kriging - Spherical	67.6768	71.2960	16.2014	262.4857
Kriging - Exponential	67.8942	71.9244	16.4513	270.6449
Kriging - Gaussian	67.4040	69.1889	10.9851	120.6721
ERA5 (reference)	61.3397	64.0828	18.8984	357.1489

Autumn IDW mean (65.91%) is 4.57% above ERA5 (61.34%). All four methods fall in Slight difference category ($D = 0.078$ to 0.149), with IDW performing best ($D=0.0782$) - the positive bias persists but the distribution shape is well-reproduced.

4.4.4. SOIL TEMPERATURE

Table 30: Seasonality Analysis of Soil Temperature (KS D values)

Season	IDW	Kriging - Spherical	Kriging - Exponential	Kriging - Gaussian
Winter (DJF)	0.3268 Large diff	0.3559 Large diff	0.3403 Large diff	0.2787 Moderate diff
Spring (MAM)	0.0972 Slight diff	0.0821 Slight diff	0.0781 Slight diff	0.1212 Slight diff
Summer (JJA)	0.0986 Slight diff	0.0662 Slight diff	0.0577 Slight diff	0.1538 Moderate diff
Autumn (SON)	0.0552 Slight diff	0.0563 Slight diff	0.0641 Slight diff	0.1151 Slight diff

Winter (DJF) :

Table 31: Winter season central tendency and dispersion parameters for Soil Temperature

Method	Mean	Median	Std dev	Variance
IDW	2.3511	2.7938	8.9651	80.3732
Kriging - Spherical	2.8565	3.0733	9.8274	96.5771

Kriging - Exponential	2.5195	2.1319	9.2109	84.8405
Kriging - Gaussian	0.0561	0.0139	5.4553	29.7605
ERA5 (reference)	3.3552	0.7832	6.1888	38.3013

Winter is anomalous: ERA5 (3.36°C) is warmer than IDW (2.35°C), the reverse of all other seasons. KS D for IDW (0.3268 - Large difference) is the worst result for soil temperature, driven by ERA5's bimodal frozen-unfrozen soil distribution that interpolation cannot reproduce.

Spring (MAM):

Table 32: Spring season central tendency and dispersion parameters for Soil Temperature

Method	Mean	Median	Std dev	Variance
IDW	14.0927	14.7913	8.6914	75.5410
Kriging - Spherical	14.3210	15.2034	9.2376	85.3334
Kriging - Exponential	14.1085	14.8650	9.0610	82.1016
Kriging - Gaussian	11.8868	12.2378	6.7115	45.0446
ERA5 (reference)	11.2761	10.8662	8.8534	78.3828

Spring IDW mean (14.09°C) is 2.81°C above ERA5 (11.28°C); the largest seasonal warm bias for soil temperature. ERA5's soil model responds more slowly to spring warming due to snowmelt and frozen ground, while station observations at lower elevations show faster warming

Summer (JJA):

Table 33: Summer season central tendency and dispersion parameters for Soil Temperature

Method	Mean	Median	Std dev	Variance
IDW	25.7027	25.9652	4.8874	23.8867
Kriging - Spherical	25.5263	25.7700	5.1776	26.8080
Kriging - Exponential	25.3314	25.5576	5.2511	27.5735
Kriging - Gaussian	23.9738	23.7705	3.2291	10.4269
ERA5 (reference)	24.0820	24.1775	6.2563	39.1414

Summer shows good mean agreement: IDW (25.70°C) is 1.62°C above ERA5 (24.08°C). Kriging Spherical achieves the best distributional match ($D = 0.0662$ - Slight difference), confirming the spherical variogram captures the summer soil temperature gradient most accurately.

Autumn (SON):

Table 34: Autumn season central tendency and dispersion parameters for Soil Temperature

Method	Mean	Median	Std dev	Variance
IDW	14.4462	15.0471	8.6996	75.6829

Kriging - Spherical	14.7566	15.6474	9.0609	82.0991
Kriging - Exponential	14.4860	15.1629	8.8620	78.5358
Kriging - Gaussian	12.1804	12.3871	7.1869	51.6517
ERA5 (reference)	14.0700	14.0721	8.5449	73.0150

Autumn shows the closest overall agreement - IDW (14.45°C) vs ERA5 (14.07°C), a difference of only 0.38°C. Variances are nearly equal (IDW 75.68, ERA5 73.02), making autumn the best season for soil temperature comparison.

4.4.5. SOIL MOISTURE

Table 35: Seasonality Analysis of Soil Moisture (KS D values)

Season	IDW	Kriging - Spherical	Kriging - Exponential	Kriging - Gaussian
Winter (DJF)	0.1563 Moderate diff	0.2220 Moderate diff	0.2072 Moderate diff	0.3088 Large diff
Spring (MAM)	0.2287 Moderate diff	0.3005 Large diff	0.2809 Moderate diff	0.3494 Large diff
Summer (JJA)	0.2275 Moderate diff	0.2859 Moderate diff	0.3059 Large diff	0.3137 Large diff
Autumn (SON)	0.2392	0.3065	0.2974	0.3259

	Moderate diff	Large diff	Moderate diff	Large diff
--	---------------	------------	---------------	------------

Winter (DJF) :

Table 36: Winter season central tendency and dispersion parameters for Soil Moisture

Method	Mean	Median	Std dev	Variance
IDW	0.2381	0.2433	0.0874	0.0076
Kriging - Spherical	0.2363	0.2362	0.0739	0.0055
Kriging - Exponential	0.2376	0.2358	0.0776	0.0060
Kriging - Gaussian	0.2435	0.2386	0.0550	0.0030
ERA5 (reference)	0.2867	0.2983	0.1038	0.0108

ERA5 (0.287 m³/m³) is consistently higher than IDW (0.238 m³/m³) by 0.049 (the smallest seasonal offset) . Winter is the best season for soil moisture distributional agreement (IDW D = 0.1563 - Moderate difference) as near-saturation conditions reduce spatial contrasts across the network.

Spring (MAM):

Table 37: Spring season central tendency and dispersion parameters for Soil Moisture

Method	Mean	Median	Std dev	Variance
IDW	0.2201	0.2217	0.0875	0.0077

Kriging - Spherical	0.2215	0.2194	0.0695	0.0048
Kriging - Exponential	0.2232	0.2180	0.0747	0.0056
Kriging - Gaussian	0.2295	0.2276	0.0553	0.0031
ERA5 (reference)	0.2737	0.2978	0.1153	0.0133

Spring ERA5 wet bias increases to 0.054 m³/m³ as the shallow ERA5 layer (0-7cm) receives more direct recharge from spring rainfall than the USCRN 10cm sensor. ERA5 variance (0.0133) is nearly triple IDW's (0.0077), reflecting the extreme wet-dry contrast ERA5 captures across the USA in spring.

Summer (JJA):

Table 38: Summer season central tendency and dispersion parameters for Soil Moisture

Method	Mean	Median	Std dev	Variance
IDW	0.1661	0.1573	0.0720	0.0052
Kriging - Spherical	0.1684	0.1624	0.0562	0.0032
Kriging - Exponential	0.1699	0.1632	0.0585	0.0034
Kriging - Gaussian	0.1720	0.1671	0.0471	0.0022

ERA5(reference)	0.2161	0.2168	0.1059	0.0112
------------------	--------	--------	--------	--------

Summer has the lowest soil moisture with IDW mean $0.166 \text{ m}^3/\text{m}^3$ vs ERA5 $0.216 \text{ m}^3/\text{m}^3$. All Kriging methods reach Large distributional difference ($D = 0.286$ to 0.314) in summer, making this the most challenging variable-season combination for Kriging interpolation

Autumn (SON):

Table 39: Autumn season central tendency and dispersion parameters for Soil Moisture

Method	Mean	Median	Std dev	Variance
IDW	0.1849	0.1798	0.0815	0.0066
Kriging - Spherical	0.1848	0.1804	0.0636	0.0040
Kriging - Exponential	0.1865	0.1800	0.0668	0.0045
Kriging - Gaussian	0.1911	0.1837	0.0566	0.0032
ERA5 (reference)	0.2424	0.2525	0.1082	0.0117

The ERA5 wet bias ($0.057 \text{ m}^3/\text{m}^3$) is consistent with all other seasons, confirming it is a structural depth mismatch rather than a seasonal artefact. IDW ($D = 0.2392$ - Moderate difference) performs best in autumn, marginally better than spring and summer.

5. MODEL SUITABILITY AND RECOMMENDATIONS

Table 40: Best Model for the various Variables

Variable	Best method			2nd Best method		
	Method	RMSE	r	Method	RMSE	r
Air temperature (°C)	Kriging - Spherical	2.8568	0.9699	IDW	3.2023	0.9624
Precipitation (mm/day)	IDW	4.2132	0.6478	Kriging - Spherical	4.2347	0.6293
Relative humidity (%)	Kriging - Spherical	10.9387	0.8405	Kriging - Exponential	11.0167	0.8391
Soil temperature (°C)	Kriging - Spherical	4.4113	0.9379	IDW	4.4810	0.9296
Soil moisture (m ³ /m ³)	Kriging - Spherical	0.0939	0.6835	IDW	0.0957	0.6607

Although Kriging Gaussian achieves the lowest RMSE for precipitation (3.72mm), it is not recommended because it produces the worst distributional agreement (KS D = 0.44 to 0.49 - Large difference across all seasons).

Air Temperature : Kriging Spherical (RMSE = 2.86°C, r = 0.97)

Air temperature has the longest spatial correlation length among all five variables, driven by large-scale pressure systems and advection. The spherical variogram model correctly represents the finite decorrelation distance of temperature, reaching its sill at a physically realistic range. Kriging Spherical achieves [KS D = 0.0300 (Very similar)] the best distributional match of any method for any variable in this study. The warm bias of approximately 3°C in winter is a

consistent feature of all methods and reflects the systematic placement of USCRN stations at lower elevations, not a model deficiency.

Precipitation : IDW (RMSE = 4.21mm, $r = 0.65$)

Daily precipitation is a zero-inflated, highly right-skewed variable with short spatial correlation lengths during convective events. Kriging methods particularly Exponential and Gaussian artificially smooth the field and inflate median values in dry grid cells. IDW preserves the zero-inflated character of the distribution more faithfully, achieving the lowest KS D across all seasons. The moderate correlation ($r = 0.65$) reflects the fundamental challenge of interpolating convective precipitation from only 130 stations, not a limitation of IDW specifically.

Relative Humidity : Kriging Spherical (RMSE = 10.94%, $r = 0.84$)

Relative humidity is a spatially smooth variable controlled by large-scale atmospheric circulation, making it well-suited to variogram-based interpolation. Kriging Spherical achieves the best distributional agreement in winter and spring ($D = 0.059$ - Slight difference). The systematic positive bias of all methods relative to ERA5 (ranging from 0.86% in winter to 7.48% in summer) reflects

Soil Temperature : Kriging Spherical (RMSE = 4.41°C, $r = 0.94$)

Soil temperature has a very long spatial correlation length due to the high thermal inertia of soil. Kriging Spherical consistently achieves Slight difference with ERA5 in all non-winter seasons ($D < 0.12$). The Large distributional difference in winter ($D = 0.356$) is specifically linked to ERA5's representation of freeze-thaw and snowmelt processes, ERA5 produces a bimodal frozen-unfrozen distribution that station interpolation cannot reproduce. The near-perfect mean agreement in autumn (IDW 14.45°C vs ERA5 14.07°C) confirms that the interpolation correctly captures the soil thermal regime outside of winter frozen-ground conditions.

Soil Moisture : Kriging Spherical (RMSE = 0.094, $r = 0.68$)

The recommendation for Kriging Spherical over IDW is marginal (RMSE difference of 0.002). The dominant feature of the soil moisture comparison is not the interpolation method but the systematic ERA5 wet bias of 0.049 to 0.057 m³/m³ across all seasons a structural mismatch between ERA5 swvl1 (0-7cm layer) and USCRN measurements at 10cm depth. This cannot be

corrected by any interpolation method. ERA5 variance consistently exceeds interpolated variance by a factor of 2 to 3, indicating ERA5 captures extreme wet-dry contrasts across the USA that the sparse station network cannot resolve.

6. APPENDIX

CODE IMPLEMENTATION

1. Mounting **Google Drive** to the Colab environment and setting a variable pointing to a specific project folder (project_kar) in MyDrive.

```
from google.colab import drive
drive.mount('/content/drive')

folder_path = '/content/drive/MyDrive/project_kar'
```

2. Install Necessary Package

```
pip install numpy pandas scipy matplotlib netCDF4 xarray
pykrige scikit-learn requests geopandas tqdm
```

3. Importing five USCRN climate datasets (air temperature, precipitation, relative humidity, soil moisture, soil temperature) for 2006–2021, converting temperature values from Kelvin to Celsius, reshaping all datasets from wide to long format, merging them into a single station_data DataFrame along with station coordinates from metadata, and cleaning invalid values.

```
import pandas as pd
import numpy as np

metadata =
pd.read_excel('All_CONUS_Station_Information_V2.xlsx')

print(metadata.columns.tolist())
print(metadata.head())

df_temp =
pd.read_excel('USCRN_AirTemperature_2006_2021.xlsx')
```

```

df_precip =
pd.read_excel('USCRN_Precipitation_2006_2021.xlsx')
df_rh      =
pd.read_excel('USCRN_RelativeHumidity_2006_2021.xlsx')
df_soilm   =
pd.read_excel('USCRN_SoilMoisture10cm_2006_2021.xlsx')
df_soilt   =
pd.read_excel('USCRN_SoilTemperature_2006_2021.xlsx')

print(df_temp['date'].iloc[0])

print(df_temp['date'].iloc[0])

station_cols = [c for c in df_temp.columns if c !=
'date']

df_temp[station_cols] = df_temp[station_cols] - 273.15
df_soilt[station_cols] = df_soilt[station_cols] - 273.15

def melt_to_long(df, var_name):
    station_cols = [c for c in df.columns if c != 'date']
    melted = df.melt(id_vars='date',
value_vars=station_cols,
                    var_name='StationName',
value_name=var_name)
    return melted

long_temp    = melt_to_long(df_temp,    'air_temperature')
long_precip  = melt_to_long(df_precip,  'precipitation')
long_rh      = melt_to_long(df_rh,
'relative_humidity')
long_soilm   = melt_to_long(df_soilm,    'soil_moisture')
long_soilt   = melt_to_long(df_soilt,    'soil_temperature')

```

```

station_data = long_temp.copy()
station_data = station_data.merge(long_precip,
on=['date', 'StationName'], how='left')
station_data = station_data.merge(long_rh,
on=['date', 'StationName'], how='left')
station_data = station_data.merge(long_soilm,
on=['date', 'StationName'], how='left')
station_data = station_data.merge(long_soilt,
on=['date', 'StationName'], how='left')

station_data = station_data.merge(
    metadata[['StationName', 'Lon', 'Lat']],
    on='StationName', how='left'
)

station_data = station_data.rename(columns={'Lon': 'lon',
'Lat': 'lat'})

station_data.loc[station_data['relative_humidity'] > 100,
'relative_humidity'] = np.nan
station_data.loc[station_data['precipitation'] < 0,
'precipitation'] = np.nan

print(f"\nDate range: {station_data['date'].min()} to
{station_data['date'].max()}")
print(f"Total rows: {len(station_data)}")
print(f"Stations:
{station_data['StationName'].unique()}")
print(station_data.head())
print(station_data[['air_temperature', 'precipitation',
'relative_humidity', 'soil_moisture',
'soil_temperature']].describe())

```

4. Defining a 2° resolution grid over the contiguous US (longitude: -125 to -67, latitude: 24 to 50) and creating a 2D meshgrid of cell-center coordinates to serve as the spatial interpolation framework.

```

lon_min, lon_max = -125, -67
lat_min, lat_max = 24, 50
resolution = 2.0 # degrees

grid_lons = np.arange(lon_min + resolution/2, lon_max,
resolution)
grid_lats = np.arange(lat_min + resolution/2, lat_max,
resolution)

grid_lon_2d, grid_lat_2d = np.meshgrid(grid_lons,
grid_lats)

print(f"Grid size: {len(grid_lats)} rows ×
{len(grid_lons)} cols")
print(f"Total grid cells: {len(grid_lats) *
len(grid_lons)}")

```

5. Saving the 1D and 2D grid coordinate arrays (grid_lons, grid_lats, grid_lon_2d, grid_lat_2d) as .npy files to Google Drive for later reuse.

```

import numpy as np

# SAVE
np.save(f'{folder_path}/grid_lons.npy', grid_lons)
np.save(f'{folder_path}/grid_lats.npy', grid_lats)
np.save(f'{folder_path}/grid_lon_2d.npy', grid_lon_2d)
np.save(f'{folder_path}/grid_lat_2d.npy', grid_lat_2d)
print("✓ Grid arrays saved!")

```

6. Defining an **Inverse Distance Weighting (IDW)** interpolation function using a KD-Tree to spatially interpolate station observations onto the 2D grid, and testing it on air temperature data for July 15, 2010.

```

from scipy.spatial import cKDTree

def idw_interpolation(lon_obs, lat_obs, values_obs,
lon_grid, lat_grid, power=2):

```

```

mask = ~np.isnan(values_obs)
lon_obs = lon_obs[mask]
lat_obs = lat_obs[mask]
values_obs = values_obs[mask]

if len(values_obs) < 3:
    return np.full(lon_grid.shape, np.nan)

obs_coords = np.column_stack([lon_obs, lat_obs])
grid_shape = lon_grid.shape
grid_coords = np.column_stack([lon_grid.ravel(),
lat_grid.ravel()])

tree = cKDTree(obs_coords)
k = min(len(values_obs), 12)
distances, indices = tree.query(grid_coords, k=k)
distances = np.where(distances == 0, 1e-10,
distances)

weights = 1.0 / distances**power
weights_sum = weights.sum(axis=1)
values_at_neighbors = values_obs[indices]
grid_values_flat = (weights *
values_at_neighbors).sum(axis=1) / weights_sum

return grid_values_flat.reshape(grid_shape)

# — Test for one day

```

```

target_date = pd.Timestamp('2010-07-15')    # must be
within 2006-2021
variable = 'air_temperature'

day_data = station_data[station_data['date'] ==
target_date].copy()

print(f"Stations available on {target_date.date()}:
{len(day_data)}")

```

```

lon_obs = day_data['lon'].values
lat_obs = day_data['lat'].values
val_obs = day_data[variable].values

grid_idw = idw_interpolation(lon_obs, lat_obs, val_obs,
grid_lon_2d, grid_lat_2d, power=2)

print(f"IDW result for {target_date.date()}, {variable}")
print(f"  Min: {np.nanmin(grid_idw):.2f} °C")
print(f"  Max: {np.nanmax(grid_idw):.2f} °C")
print(f"  Mean: {np.nanmean(grid_idw):.2f} °C")

```

7. Implementing **Ordinary Kriging** interpolation (with spherical, exponential, and gaussian variogram models) across all 5 climate variables for every day from 2006–2021, with checkpoint-based crash recovery every 100 days, and saving the final 3D gridded results to Google Drive.

```

from pykrige.ok import OrdinaryKriging
from scipy.spatial import cKDTree
from tqdm import tqdm
import pickle
import os
import numpy as np
import pandas as pd

date_range = pd.date_range(start='2006-01-01',
end='2021-12-31', freq='D')
print(f"✓ date_range: {date_range[0].date()} to
{date_range[-1].date()} "
      f"({len(date_range)} days)")

VARIOGRAM_PARAMS = {
    'spherical': None,
    'exponential': None,

```



```

    'gaussian':      {'nugget': 0.5, 'sill': 1.0, 'range':
5.0},
    }

def kriging_interpolation(lon_obs, lat_obs, values_obs,
lon_grid, lat_grid,
                           variogram_model='spherical'):
    mask = ~np.isnan(values_obs)
    lon_obs     = lon_obs[mask]
    lat_obs     = lat_obs[mask]
    values_obs  = values_obs[mask]

    if len(values_obs) < 5:
        return np.full(lon_grid.shape, np.nan)

    v_mean = np.mean(values_obs)
    v_std  = np.std(values_obs) if np.std(values_obs) > 0
else 1.0
    values_norm = (values_obs - v_mean) / v_std

    try:
        params = VARIOGRAM_PARAMS.get(variogram_model,
None)

        OK = OrdinaryKriging(
            lon_obs, lat_obs, values_norm,
            variogram_model=variogram_model,
            variogram_parameters=params,
            verbose=False,
            enable_plotting=False,
            nlags=6,
            weight=True,
        )
        z, _ = OK.execute('grid', lon_grid[0, :],
lat_grid[:, 0])
        return np.array(z) * v_std + v_mean

    except Exception as e:

```

```

        print(f"    Kriging [{variogram_model}] failed:
{e}")
        return np.full(lon_grid.shape, np.nan)

variogram_models = ['spherical', 'exponential',
'gaussian']
variables = ['air_temperature', 'precipitation',
            'relative_humidity', 'soil_temperature',
'soil_moisture']

n_lats  = len(grid_lats)
n_lons  = len(grid_lons)
n_dates = len(date_range)

CHECKPOINT_FILE = f'{folder_path}/kriging_checkpoint.pkl'
SAVE_EVERY      = 100

if os.path.exists(CHECKPOINT_FILE):
    print("🔄 Checkpoint found – resuming from where it
crashed...")
    with open(CHECKPOINT_FILE, 'rb') as f:
        checkpoint = pickle.load(f)
        results_kriging_all = checkpoint['results']
        start_idx          = checkpoint['last_saved_idx'] +
1
        print(f"    Resuming from day index {start_idx} /
{n_dates}")
        print(f"    ({date_range[start_idx].date()} onwards)")
else:
    print("NEW No checkpoint found – starting fresh...")
    results_kriging_all = {
        model: {var: np.full((n_dates, n_lats, n_lons),
np.nan) for var in variables}
        for model in variogram_models
    }
    start_idx = 0

```

```

print(f"\nTotal days to process: {n_dates - start_idx}
remaining\n")

for t_idx in tqdm(range(start_idx, n_dates),
desc='Kriging all models'):

    date    = date_range[t_idx]
    day_df = station_data[station_data['date'] == date]

    if len(day_df) == 0:
        continue

    for var in variables:
        lon_obs = day_df['lon'].values
        lat_obs = day_df['lat'].values
        val_obs = day_df[var].values

        for model in variogram_models:
            results_kriging_all[model][var][t_idx] =
kriging_interpolation(
                lon_obs, lat_obs, val_obs,
                grid_lon_2d, grid_lat_2d,
                variogram_model=model
            )

    if (t_idx + 1) % SAVE_EVERY == 0:
        checkpoint = {
            'results':      results_kriging_all,
            'last_saved_idx': t_idx,
        }
        with open(CHECKPOINT_FILE, 'wb') as f:
            pickle.dump(checkpoint, f)
        print(f"\n💾 Checkpoint saved at day
{t_idx+1}/{n_dates} "
              f"({date_range[t_idx].date()})")

```

```

with open(f'{folder_path}/results_kriging_all.pkl', 'wb')
as f:
    pickle.dump(results_kriging_all, f)

if os.path.exists(CHECKPOINT_FILE):
    os.remove(CHECKPOINT_FILE)

print("\n✅ All Kriging models saved!")
print(f"    Models      : {variogram_models}")
print(f"    Variables   : {variables}")
print(f"    Shape       : ({n_dates}, {n_lats}, {n_lons})
per variable")

```

8. Running **IDW interpolation** across all 5 climate variables for every day from 2006–2021, storing the results as 3D arrays (days × lats × lons) in a dictionary results_idw.

```

from scipy.spatial import cKDTree
from tqdm import tqdm
import numpy as np
import pandas as pd
import pickle

date_range = pd.date_range(start='2006-01-01',
end='2021-12-31', freq='D')
print(f"✅ date_range: {date_range[0].date()} to
{date_range[-1].date()} ({len(date_range)} days)")

def idw_interpolation(lon_obs, lat_obs, values_obs,
lon_grid, lat_grid, power=2):
    mask = ~np.isnan(values_obs)
    lon_obs = lon_obs[mask]
    lat_obs = lat_obs[mask]
    values_obs = values_obs[mask]

    if len(values_obs) < 3:
        return np.full(lon_grid.shape, np.nan)

```

```

    obs_coords = np.column_stack([lon_obs, lat_obs])
    grid_coords = np.column_stack([lon_grid.ravel(),
lat_grid.ravel()])

    tree = cKDTree(obs_coords)
    k = min(len(values_obs), 12)
    distances, indices = tree.query(grid_coords, k=k)
    distances = np.where(distances == 0, 1e-10,
distances)

    weights = 1.0 / distances**power
    weights_sum = weights.sum(axis=1)
    values_at_neighbors = values_obs[indices]
    grid_values_flat = (weights *
values_at_neighbors).sum(axis=1) / weights_sum

    return grid_values_flat.reshape(lon_grid.shape)

variables = ['air_temperature', 'precipitation',
            'relative_humidity', 'soil_temperature',
'soil_moisture']

n_lats = len(grid_lats)
n_lons = len(grid_lons)
n_dates = len(date_range)

results_idw = {
    var: np.full((n_dates, n_lats, n_lons), np.nan)
    for var in variables
}

for t_idx, date in enumerate(tqdm(date_range, desc='IDW
interpolation')):

    day_df = station_data[station_data['date'] == date]

```

```

if len(day_df) == 0:
    continue

for var in variables:
    lon_obs = day_df['lon'].values
    lat_obs = day_df['lat'].values
    val_obs = day_df[var].values

    results_idw[var][t_idx] = idw_interpolation(
        lon_obs, lat_obs, val_obs,
        grid_lon_2d, grid_lat_2d,
        power=2
    )

print("\n✅ IDW interpolation done!")
print(f"    Variables : {variables}")
print(f"    Shape      : ({n_dates}, {n_lats}, {n_lons})
per variable")

```

9. Converting the IDW and all three Kriging interpolation results into **NetCDF datasets** using xarray (with proper time, lat, lon coordinates and metadata attributes) and saving them as .nc files to Google Drive.

```

import xarray as xr

def save_to_netcdf(results_dict, grid_lons, grid_lats,
date_range,
                    method_name, output_path):
    data_vars = {}
    for var, arr in results_dict.items():
        data_vars[var] = xr.DataArray(
            data=arr,
            dims=['time', 'lat', 'lon'],
            attrs={
                'long_name': var.replace('_', ' '),
            }).title(),
            'interpolation_method': method_name,

```

```

        'source': 'USCRN station observations',
    }
)
ds = xr.Dataset(
    data_vars=data_vars,
    coords={
        'time': ('time', date_range),
        'lat': ('lat', grid_lats),
        'lon': ('lon', grid_lons),
    },
    attrs={
        'title': f'Gridded Hydroclimatic Variables - {method_name}',
        'spatial_resolution': '2 degree',
        'temporal_resolution': 'daily',
        'domain': 'Conterminous USA',
        'created_by': 'GNR 640 Mini Project',
    }
)
ds.to_netcdf(output_path)
print(f"✓ Saved: {output_path}")
return ds

# Save IDW
ds_idw = save_to_netcdf(
    results_idw, grid_lons, grid_lats, date_range,
    method_name='IDW_power2',
    output_path=f'{folder_path}/gridded_USA_IDW.nc'
)

# Save all 3 Kriging models
for model in ['spherical', 'exponential', 'gaussian']:
    save_to_netcdf(
        results_kriging_all[model], grid_lons, grid_lats,
        date_range,
        method_name=f'Kriging_{model}',

```

```

output_path=f'{folder_path}/gridded_USA_Kriging_{model}.nc'
    )

print("\n✓ All files saved to Drive!")

```

10. Downloading **ERA5-Land reanalysis data** for 2011–2012 from the Copernicus CDS API - temperature, dewpoint, soil temperature, and soil moisture at all 24 hourly timesteps (for daily averaging), and precipitation at all 24 hourly timesteps (for daily summation), saved as monthly .nc files to Google Drive with skip-if-exists logic.

```

import cdsapi
import os

c = cdsapi.Client()

folder_path = '/content/drive/MyDrive/project_kar'

all_24_hours = [f'{h:02d}:00' for h in range(24)] #
['00:00', '01:00', ..., '23:00']

for year in range(2011, 2013):
    for month in range(1, 13):

        output_file =
f'{folder_path}/era5_mean_{year}_{month:02d}.nc'

        if os.path.exists(output_file):
            print(f"Mean {year}-{month:02d} already
exists, skipping...")
            continue

        print(f"Downloading mean variables -
{year}-{month:02d}...")

        c.retrieve(
            'reanalysis-era5-land',

```



```

        {
            'variable': [
                '2m_temperature',
                '2m_dewpoint_temperature',
                'soil_temperature_level_1',
                'volumetric_soil_water_layer_1',
            ],
            'year': [str(year)],
            'month': [f'{month:02d}'],
            'day': [f'{d:02d}' for d in range(1,
32)],

            'time': all_24_hours, # all 24 hours
            'area': [50, -125, 24, -67],
            'format': 'netcdf',
        },
        output_file
    )
    print(f"✓ Saved:
era5_mean_{year}_{month:02d}.nc")

print("\n✓ All MEAN downloads done!\n")

for year in range(2011, 2013):
    for month in range(1, 13):

        output_file =
f'{folder_path}/era5_precip_{year}_{month:02d}.nc'

        if os.path.exists(output_file):
            print(f"Precip {year}-{month:02d} already
exists, skipping...")
            continue

        print(f"Downloading precipitation -
{year}-{month:02d}...")

        c.retrieve(

```

```

        'reanalysis-era5-land',
        {
            'variable': ['total_precipitation'],
            'year': [str(year)],
            'month': [f'{month:02d}'],
            'day': [f'{d:02d}' for d in range(1,
32)],
            'time': all_24_hours,
            'area': [50, -125, 24, -67],
            'format': 'netcdf',
        },
        output_file
    )
    print(f"✓ Saved:
era5_precip_{year}_{month:02d}.nc")

print("\n✓ All PRECIP downloads done!")
print("\nSummary:")
print("  era5_mean_YYYY_MM.nc → t2m, d2m, stl1, swvl1
at all 24 hours (average daily)")
print("  era5_precip_YYYY_MM.nc → tp at all 24 hours (sum
for daily total)")

```

11. Extracting the downloaded ERA5 monthly .nc files (which were stored as zips) into separate folders (era5_mean_extracted/ and era5_precip_extracted/), renaming each extracted file with its corresponding year-month identifier, and verifying the counts and filenames of all extracted files.

```

!pip install netcdf4 pykrige --quiet

import numpy as np
import pandas as pd
import scipy.stats as stats
import xarray as xr
import pickle
import os
import glob
print("Libraries loaded!")

```

```

import zipfile
import os
import glob
import shutil

folder_path = '/content/drive/MyDrive/project_kar'

mean_zips =
sorted(glob.glob(f'{folder_path}/era5_mean_*.nc'))
print(f"\nExtracting {len(mean_zips)} mean zip files...")

for zip_path in mean_zips:

    basename = os.path.basename(zip_path)
    year_month = basename.replace('era5_mean_',
    '').replace('.nc', '') # 2006_01

    try:
        with zipfile.ZipFile(zip_path, 'r') as z:
            for name in z.namelist():
                # Extract to temp location
                z.extract(name,
f'{folder_path}/era5_mean_extracted/')
                # Rename to include year_month
                old_path =
f'{folder_path}/era5_mean_extracted/{name}'
                new_path =
f'{folder_path}/era5_mean_extracted/era5_mean_{year_month}
.nc'

                os.rename(old_path, new_path)
                print(f"✓ {basename} →
era5_mean_{year_month}.nc")
            except Exception as e:
                print(f"✗ Failed: {basename} - {e}")

print(f"\n✓ All mean files extracted!\n")

```

```

precip_zips =
sorted(glob.glob(f'{folder_path}/era5_precip_*.nc'))
print(f"Extracting {len(precip_zips)} precip zip
files...")

for zip_path in precip_zips:
    basename = os.path.basename(zip_path)
    year_month = basename.replace('era5_precip_',
    '').replace('.nc', '')

    try:
        with zipfile.ZipFile(zip_path, 'r') as z:
            for name in z.namelist():
                z.extract(name,
f'{folder_path}/era5_precip_extracted/')
                old_path =
f'{folder_path}/era5_precip_extracted/{name}'
                new_path =
f'{folder_path}/era5_precip_extracted/era5_precip_{year_m
onth}.nc'
                os.rename(old_path, new_path)
                print(f"✓ {basename} →
era5_precip_{year_month}.nc")
            except Exception as e:
                print(f"✗ Failed: {basename} - {e}")

print("\n✓ All precip files extracted!")

# — Verify

mean_nc =
sorted(glob.glob(f'{folder_path}/era5_mean_extracted/*.nc
'))
precip_nc =
sorted(glob.glob(f'{folder_path}/era5_precip_extracted/*.
nc'))
print(f"\nExtracted mean files : {len(mean_nc)}")
print(f"Extracted precip files: {len(precip_nc)}")

```

```

print(f"\nFirst mean file :
{os.path.basename(mean_nc[0])}")
print(f"Last mean file :
{os.path.basename(mean_nc[-1])}")
print(f"First precip file:
{os.path.basename(precip_nc[0])}")
print(f"Last precip file :
{os.path.basename(precip_nc[-1])}")

```

12. Loading all extracted ERA5 monthly NetCDF files into two combined xarray datasets (era5_mean and era5_precip) using open_mfdataset, renaming valid_time to time if needed, and printing the variable names, time range, and shape of each dataset.

```

import xarray as xr
import glob

mean_files =
sorted(glob.glob(f'{folder_path}/era5_mean_extracted/*.nc
'))
print(f"Found {len(mean_files)} mean files")

era5_mean = xr.open_mfdataset(
    mean_files,
    combine='by_coords',
    engine='netcdf4',
    parallel=False
)

if 'valid_time' in era5_mean.dims:
    era5_mean = era5_mean.rename({'valid_time': 'time'})

print("✓ ERA5 mean loaded!")
print(f" Variables : {list(era5_mean.data_vars)}")
print(f" Time range:
{str(era5_mean.time.values[0])[:10]} to "
      f"{str(era5_mean.time.values[-1])[:10]}")
print(f" Shape      : {dict(era5_mean.sizes)}")

```

```

precip_files =
sorted(glob.glob(f'{folder_path}/era5_precip_extracted/*.
nc'))
print(f"\nFound {len(precip_files)} precip files")

era5_precip = xr.open_mfdataset(
    precip_files,
    combine='by_coords',
    engine='netcdf4',
    parallel=False
)

if 'valid_time' in era5_precip.dims:
    era5_precip = era5_precip.rename({'valid_time':
'time'})

print("✓ ERA5 precip loaded!")
print(f"  Variables : {list(era5_precip.data_vars)}")
print(f"  Time range:
{str(era5_precip.time.values[0])[:10]} to "
      f"{str(era5_precip.time.values[-1])[:10]}")
print(f"  Shape      : {dict(era5_precip.sizes)}")

print("\n✓ All ERA5 files loaded successfully!")

```

13. Aggregating ERA5 hourly data to daily means, converting units (Kelvin → °C, dewpoint → relative humidity via Magnus formula, precipitation m → mm), clipping to the CONUS domain, coarsening to 2° resolution, and interpolating onto the project's standard grid for alignment with the interpolated USCRN data.

```

import numpy as np
import xarray as xr

print("Processing ERA5 to daily values...")

```

```

era5_daily_mean = era5_mean.resample(time='1D').mean()
print("✓ Mean variables aggregated to daily mean")
print(f"  Shape: {dict(era5_daily_mean.sizes)}")

era5_daily_mean['t2m_degC'] = era5_daily_mean['t2m'] -
273.15
era5_daily_mean['soil_temp_C'] = era5_daily_mean['st11']
- 273.15

T_C = era5_daily_mean['t2m'] - 273.15
Td_C = era5_daily_mean['d2m'] - 273.15
era5_daily_mean['relative_humidity'] = (
    100 * np.exp((17.625 * Td_C) / (243.04 + Td_C))
    / np.exp((17.625 * T_C) / (243.04 + T_C))
).clip(0, 100)

print("✓ Unit conversions done!")
print("  t2m → t2m_degC (°C)")
print("  st11 → soil_temp_C (°C)")
print("  t2m + d2m → relative_humidity (%)")

era5_daily_precip = era5_precip.copy()
era5_daily_precip['precip_mm'] = era5_daily_precip['tp']
* 1000
print("✓ Precipitation converted m → mm")

era5_mean_usa = era5_daily_mean.sel(
    latitude=slice(50, 24),
    longitude=slice(-125, -67)
)
era5_precip_usa = era5_daily_precip.sel(
    latitude=slice(50, 24),
    longitude=slice(-125, -67)
)

era5_mean_coarse = era5_mean_usa.coarsen(
    latitude=20, longitude=20, boundary='trim'
).mean()

```

```

era5_precip_coarse = era5_precip_usa.coarsen(
    latitude=20, longitude=20, boundary='trim'
).mean()

print("✓ Coarsened to 2° resolution")
print(f"  Mean shape   : {dict(era5_mean_coarse.sizes)}")
print(f"  Precip shape:
{dict(era5_precip_coarse.sizes)}")

era5_mean_coarse = era5_mean_coarse.rename({'latitude':
'lat', 'longitude': 'lon'})
era5_precip_coarse =
era5_precip_coarse.rename({'latitude': 'lat',
'longitude': 'lon'})

era5_mean_2deg = era5_mean_coarse.interp(
    lat=xr.DataArray(grid_lats, dims='lat'),
    lon=xr.DataArray(grid_lons, dims='lon'),
    method='linear'
)

era5_precip_2deg = era5_precip_coarse.interp(
    lat=xr.DataArray(grid_lats, dims='lat'),
    lon=xr.DataArray(grid_lons, dims='lon'),
    method='linear'
)

print("✓ Aligned to interpolation grid")
print(f"  ERA5 lats : {era5_mean_2deg.lat.values}")
print(f"  Your lats : {grid_lats}")
print(f"  Match      :
{np.allclose(era5_mean_2deg.lat.values, grid_lats)}")

print("\n✅ ERA5 processing complete!")
print(f"  Mean variables :
{list(era5_mean_2deg.data_vars)}")
print(f"  Precip variable:
{list(era5_precip_2deg.data_vars)}")

```


14. Processing ERA5 data year-by-year (2006–2021) to avoid memory overload — each year's mean and precip files are loaded, aggregated, unit-converted, clipped to CONUS, coarsened to 2°, and saved individually as compressed NetCDF files, then all yearly files are combined into two final datasets (era5_mean_2deg_daily.nc and era5_precip_2deg_daily.nc).

```
import xarray as xr
import numpy as np
import os
import glob

print("Processing and saving ERA5 year by year...\n")

folder_path = '/content/drive/MyDrive/project_kar'

# — Create temp folder for yearly files
_____

os.makedirs(f'{folder_path}/era5_yearly_processed',
            exist_ok=True)

for year in range(2006, 2022):

    # Skip if already processed
    mean_done =
os.path.exists(f'{folder_path}/era5_yearly_processed/mean_
_{year}.nc')
    precip_done =
os.path.exists(f'{folder_path}/era5_yearly_processed/prec
ip_{year}.nc')

    if mean_done and precip_done:
        print(f"✓ {year} already processed, skipping...")
        continue

    print(f"Processing {year}...")
```

```

# — Load just this year

mean_files = sorted(glob.glob(
f'{folder_path}/era5_mean_extracted/era5_mean_{year}_*.nc'
))
precip_files = sorted(glob.glob(
f'{folder_path}/era5_precip_extracted/era5_precip_{year}_*.nc'
))

ds_mean = xr.open_mfdataset(
    mean_files, combine='by_coords',
    engine='netcdf4', parallel=False
)
if 'valid_time' in ds_mean.dims:
    ds_mean = ds_mean.rename({'valid_time': 'time'})

ds_precip = xr.open_mfdataset(
    precip_files, combine='by_coords',
    engine='netcdf4', parallel=False
)
if 'valid_time' in ds_precip.dims:
    ds_precip = ds_precip.rename({'valid_time':
'time'})

# — Process mean

ds_mean_daily =
ds_mean.resample(time='1D').mean().compute()
ds_mean.close()    # free memory immediately

ds_mean_daily['t2m_degC'] = ds_mean_daily['t2m'] -
273.15
ds_mean_daily['soil_temp_C'] = ds_mean_daily['st11']
- 273.15

```

```

T_C = ds_mean_daily['t2m'] - 273.15
Td_C = ds_mean_daily['d2m'] - 273.15
ds_mean_daily['relative_humidity'] = (
    100 * np.exp((17.625 * Td_C) / (243.04 + Td_C))
    / np.exp((17.625 * T_C) / (243.04 + T_C))
).clip(0, 100)

ds_mean_usa = ds_mean_daily.sel(
    latitude=slice(50, 24), longitude=slice(-125,
-67)
)
ds_mean_coarse = ds_mean_usa.coarsen(
    latitude=20, longitude=20, boundary='trim'
).mean()
ds_mean_coarse = ds_mean_coarse.rename({'latitude':
'lat', 'longitude': 'lon'})
ds_mean_2deg = ds_mean_coarse.interp(
    lat=xr.DataArray(grid_lats, dims='lat'),
    lon=xr.DataArray(grid_lons, dims='lon'),
    method='linear'
)

# — Save mean immediately — do NOT keep in RAM


---


ds_mean_2deg.to_netcdf(
f'{folder_path}/era5_yearly_processed/mean_{year}.nc',
    encoding={var: {'zlib': True, 'complevel': 1}
    for var in ds_mean_2deg.data_vars}
)
del ds_mean_daily, ds_mean_usa, ds_mean_coarse,
ds_mean_2deg
print(f"    ✓ Mean {year} saved")

# — Process precip


---


ds_precip_loaded = ds_precip.compute()
ds_precip.close()

```

```

    ds_precip_loaded['precip_mm'] =
ds_precip_loaded['tp'] * 1000
    ds_precip_usa = ds_precip_loaded.sel(
        latitude=slice(50, 24), longitude=slice(-125,
-67)
    )
    ds_precip_coarse = ds_precip_usa.coarsen(
        latitude=20, longitude=20, boundary='trim'
    ).mean()
    ds_precip_coarse =
ds_precip_coarse.rename({'latitude': 'lat', 'longitude':
'lon'})
    ds_precip_2deg = ds_precip_coarse.interp(
        lat=xr.DataArray(grid_lats, dims='lat'),
        lon=xr.DataArray(grid_lons, dims='lon'),
        method='linear'
    )

# — Save precip immediately


---


    ds_precip_2deg.to_netcdf(
f'{folder_path}/era5_yearly_processed/precip_{year}.nc',
        encoding={var: {'zlib': True, 'complevel': 1}
            for var in ds_precip_2deg.data_vars}
    )
    del ds_precip_loaded, ds_precip_usa,
ds_precip_coarse, ds_precip_2deg
    print(f" ✓ Precip {year} saved")

print("\n✅ All years processed and saved!")

# — Combine all yearly files into final files


---


print("\nCombining all years into final files...")

```

```

mean_yearly =
sorted(glob.glob(f'{folder_path}/era5_yearly_processed/me
an_*.nc'))
precip_yearly =
sorted(glob.glob(f'{folder_path}/era5_yearly_processed/pr
ecip_*.nc'))

era5_mean_final = xr.open_mfdataset(
    mean_yearly, combine='by_coords', engine='netcdf4'
)
era5_mean_final.to_netcdf(
    f'{folder_path}/era5_mean_2deg_daily.nc',
    encoding={var: {'zlib': True, 'complevel': 1}
              for var in era5_mean_final.data_vars}
)
print("✓ Saved: era5_mean_2deg_daily.nc")

era5_precip_final = xr.open_mfdataset(
    precip_yearly, combine='by_coords', engine='netcdf4'
)
era5_precip_final.to_netcdf(
    f'{folder_path}/era5_precip_2deg_daily.nc',
    encoding={var: {'zlib': True, 'complevel': 1}
              for var in era5_precip_final.data_vars}
)
print("✓ Saved: era5_precip_2deg_daily.nc")

print("\n✓ Final files ready!")

```

15. Comparing all four interpolation methods (IDW + 3 Kriging variants) against ERA5 as reference - computing descriptive statistics (mean, median, std, variance, min, max) and performance metrics (RMSE, Pearson r, KS test) for each of the 5 climate variables, both over the full 2006–2021 period and broken down by season.

```

import numpy as np
import scipy.stats as stats

var_mapping = {

```

```

    'air_temperature':      ('mean',      't2m_degC'),
    'precipitation':       ('precip',    'precip_mm'),
    'relative_humidity':   ('mean',      'relative_humidity'),
    'soil_temperature':    ('mean',      'soil_temp_C'),
    'soil_moisture':       ('mean',      'swvl1'),
}

season_order = ['DJF (Winter)', 'MAM (Spring)', 'JJA
(Summer)', 'SON (Autumn)']
seasons = [get_season(d.month) for d in date_range]

methods = {
    'IDW':                      results_idw,
    'Kriging_Spherical':
results_kriging_all['spherical'],
    'Kriging_Exponential':
results_kriging_all['exponential'],
    'Kriging_Gaussian':
    results_kriging_all['gaussian'],
}

for var, (source, era5_var) in var_mapping.items():

    if source == 'mean':
        era5_vals_full = era5_mean_2deg[era5_var].values
    else:
        era5_vals_full =
era5_precip_2deg[era5_var].values

    print(f"\n{'='*70}")
    print(f"VARIABLE: {var}")
    print(f"{'='*70}")

    # — Overall statistics (full period)

    print(f"\n--- OVERALL (Full Period 2006-2021) ---")
    print(f"{'Method':22s} | {'Mean':>8} | {'Median':>8}
| {'Std':>8} | {'Variance':>10} | {'Min':>8} |
{'Max':>8}")

```

```

print("-" * 85)

for method_name, results in methods.items():
    arr = results[var].ravel()
    arr = arr[~np.isnan(arr)]
    print(f"{method_name:22s} | {np.mean(arr):>8.4f} | "
          f"{np.std(arr):>8.4f} | "
          f"{np.var(arr):>10.4f} | "
          f"{np.min(arr):>8.4f} | "
          f"{np.max(arr):>8.4f}")

    era5_arr = era5_vals_full.ravel()
    era5_arr = era5_arr[~np.isnan(era5_arr)]
    print(f"{'ERA5':22s} | {np.mean(era5_arr):>8.4f} | "
          f"{np.std(era5_arr):>8.4f} | "
          f"{np.var(era5_arr):>10.4f} | "
          f"{np.min(era5_arr):>8.4f} | "
          f"{np.max(era5_arr):>8.4f}")

# — Performance metrics (full period)

print(f"\n--- PERFORMANCE vs ERA5 (Full Period) ---")
print(f"{'Method':22s} | {'RMSE':>8} | {'r':>8} | "
      f"{'KS D':>8} | {'Similarity':>20}")
print("-" * 75)

for method_name, results in methods.items():
    pred = results[var].ravel()
    ref = era5_vals_full.ravel()
    mask = ~(np.isnan(pred) | np.isnan(ref))
    pred_c = pred[mask]; ref_c = ref[mask]
    rmse = np.sqrt(np.mean((pred_c - ref_c)**2))
    r, _ = stats.pearsonr(pred_c, ref_c)
    d, _ = stats.ks_2samp(pred_c, ref_c)
    sim = 'Very similar' if d<0.05 else 'Slight diff'
    if d<0.15 else 'Moderate diff' if d<0.30 else 'Large
diff'

```

```

        print(f"{method_name:22s} | {rmse:>8.4f} |
{r:>8.4f} | {d:>8.4f} | {sim:>20}")

# — Seasonal breakdown

for season in season_order:
    s_idx = [i for i, s in enumerate(seasons) if s ==
season]

    print(f"\n--- {season} ---")
    print(f"{'Method':22s} | {'Mean':>8} |
{'Median':>8} | {'Std':>8} | {'Variance':>10} |
{'Min':>8} | {'Max':>8}")
    print("-" * 85)

    for method_name, results in methods.items():
        arr = results[var][s_idx].ravel()
        arr = arr[~np.isnan(arr)]
        print(f"{method_name:22s} |
{np.mean(arr):>8.4f} | {np.median(arr):>8.4f} | "
              f"{np.std(arr):>8.4f} | "
              f"{np.var(arr):>10.4f} | "
              f"{np.min(arr):>8.4f} |
{np.max(arr):>8.4f}")

        era5_s = era5_vals_full[s_idx].ravel()
        era5_s = era5_s[~np.isnan(era5_s)]
        print(f"{'ERA5':22s} | {np.mean(era5_s):>8.4f} |
{np.median(era5_s):>8.4f} | "
              f"{np.std(era5_s):>8.4f} | "
              f"{np.var(era5_s):>10.4f} | "
              f"{np.min(era5_s):>8.4f} |
{np.max(era5_s):>8.4f}")

    print(f"\n{'Method':22s} | {'RMSE':>8} | {'r':>8}
| {'KS D':>8} | {'Similarity':>20}")
    print("-" * 75)

    for method_name, results in methods.items():

```



```

        pred = results[var][s_idx].ravel()
        ref = era5_vals_full[s_idx].ravel()
        mask = ~(np.isnan(pred) | np.isnan(ref))
        pred_c = pred[mask]; ref_c = ref[mask]
        rmse = np.sqrt(np.mean((pred_c - ref_c)**2))
        r, _ = stats.pearsonr(pred_c, ref_c)
        d, _ = stats.ks_2samp(pred_c, ref_c)
        sim = 'Very similar' if d<0.05 else 'Slight
diff' if d<0.15 else 'Moderate diff' if d<0.30 else
'Large diff'
        print(f"{method_name:22s} | {rmse:>8.4f} |
{r:>8.4f} | {d:>8.4f} | {sim:>20}")

```

16. Generating and saving all project figures (360 DPI) across 5 organized subfolders :
station location map, interpolation grid, variogram curves, RMSE/correlation bar charts,
scatter plots, box plots, CDF comparisons, KS heatmap, seasonal mean bar charts,
seasonal spatial maps, and method comparison maps for all 5 climate variables.

```

"""
GNR 640 — Complete Figure Generation Code
Run this in your Colab notebook after loading all
variables.
All figures saved at 360 DPI in organized subfolders.
"""

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.ticker as mticker
from matplotlib.colors import TwoSlopeNorm
from scipy.stats import pearsonr
import os
import warnings
warnings.filterwarnings('ignore')

# — Folder structure

```

```

BASE = f'{folder_path}/figures'
FOLDERS = {
    'data': f'{BASE}/01_data_sources',
    'method': f'{BASE}/02_methodology',
    'performance': f'{BASE}/03_performance',
    'stats': f'{BASE}/04_statistical_analysis',
    'seasonal': f'{BASE}/05_seasonality',
}
for f in FOLDERS.values():
    os.makedirs(f, exist_ok=True)

DPI = 360
FSIZE = (12, 7)

# Consistent color palette
METHOD_COLORS = {
    'IDW': '#2166AC',
    'Kriging Spherical': '#4DAC26',
    'Kriging Exponential': '#D7191C',
    'Kriging Gaussian': '#F4A582',
    'ERA5': '#636363',
}
SEASON_COLORS = ['#2C7BB6', '#5AAE61', '#D73027',
                  '#F4A742']
SEASON_LABELS = ['DJF (Winter)', 'MAM (Spring)', 'JJA
                  (Summer)', 'SON (Autumn)']
VARIABLES = ['air_temperature', 'precipitation',
              'relative_humidity',
              'soil_temperature', 'soil_moisture']
VAR_LABELS = ['Air Temperature', 'Precipitation',
              'Relative Humidity', 'Soil
              Temperature', 'Soil Moisture']
VAR_UNITS = ['°C', 'mm/day', '%', '°C', 'm³/m³']
VAR_CMAPS = ['RdYlBu_r', 'YlGnBu', 'BuGn',
              'RdYlBu_r', 'YlOrBr_r']

plt.rcParams.update({
    'font.family': 'DejaVu Sans',
    'font.size': 11,

```

```

        'axes.titlesize': 13,
        'axes.labelsize': 11,
        'axes.spines.top': False,
        'axes.spines.right': False,
        'figure.dpi': 100,
    ))

def save(fig, folder, filename):
    path = os.path.join(FOLDERS[folder], filename)
    fig.savefig(path, dpi=DPI, bbox_inches='tight',
                facecolor='white', edgecolor='none')
    plt.close(fig)
    print(f'    ✓ Saved: {filename}')

def get_season(month):
    if month in [12, 1, 2]: return 'DJF (Winter)'
    elif month in [3, 4, 5]: return 'MAM (Spring)'
    elif month in [6, 7, 8]: return 'JJA (Summer)'
    else: return 'SON (Autumn)'

import pandas as pd
date_range_local = pd.date_range('2006-01-01',
                                  '2021-12-31', freq='D')
seasons_list = [get_season(d.month) for d in
                 date_range_local]

ERA5_VARS = {
    'air_temperature': ('mean', 't2m_degC'),
    'precipitation': ('precip', 'precip_mm'),
    'relative_humidity': ('mean', 'relative_humidity'),
    'soil_temperature': ('mean', 'soil_temp_C'),
    'soil_moisture': ('mean', 'swvl1'),
}

def get_era5(var):
    source, key = ERA5_VARS[var]
    if source == 'mean':
        return era5_mean_2deg[key].values
    return era5_precip_2deg[key].values

```

```

print('=' * 60)
print('GMR 640 – Generating all figures at 360 DPI')
print('=' * 60)

#
=====

# SECTION 1.2 – DATA SOURCES
#
=====

print('\n[1/5] Section 1.2 – Data Sources')

# Fig 1.2.1 – USCRN Station Location Map
# Fig 1.2.1 – USCRN Station Location Map
fig, ax = plt.subplots(figsize=(14, 8))
ax.set_facecolor('#D6EAF8')
ax.set_xlim(-130, -64); ax.set_ylim(22, 52)
ax.set_xlabel('Longitude (°)', fontsize=11)
ax.set_ylabel('Latitude (°)', fontsize=11)

# FIX: use paired lon/lat per station, not separate
unique values
station_coords = station_data[['lon',
'lat']].drop_duplicates()
lons_s = station_coords['lon'].values
lats_s = station_coords['lat'].values

ax.scatter(lons_s, lats_s,
           c='#D73027', s=35, marker='^', zorder=5,
           edgecolors='#7B0000', linewidths=0.4,
           label=f'USCRN Stations (n={len(lons_s)})')

ax.legend(loc='lower right', fontsize=10, framealpha=0.9)
ax.grid(True, alpha=0.3, linestyle='--')

```

```

ax.set_title('Fig. 1.2.1 USCRN Station Network –
Contiguous United States\n'
            'Station locations used as input for spatial
interpolation (2006–2021)',
            pad=12, fontsize=12)
save(fig, 'data', 'Fig1_2_1_USCRN_Station_Locations.png')

#
=====

# SECTION 1.3 – METHODOLOGY
#
=====

print('\n[2/5] Section 1.3 – Methodology')

# Fig 1.3.1 – 2-degree interpolation grid
fig, ax = plt.subplots(figsize=(14, 8))
ax.set_facecolor('#D6EAF8')
ax.set_xlim(-128, -64); ax.set_ylim(22, 52)
ax.set_xlabel('Longitude (°)', fontsize=11)
ax.set_ylabel('Latitude (°)', fontsize=11)
for lon in grid_lons:
    ax.axvline(lon, color='#2166AC', lw=0.6, alpha=0.5)
for lat in grid_lats:
    ax.axhline(lat, color='#2166AC', lw=0.6, alpha=0.5)
ax.scatter([l for l in grid_lons for _ in grid_lats],
           [la for _ in grid_lons for la in grid_lats],
           c='#2166AC', s=40, zorder=4, label='Grid node
(2° × 2°)')
# FIX: use paired coords here too
station_coords = station_data[['lon',
'lat']].drop_duplicates()
lons_s = station_coords['lon'].values
lats_s = station_coords['lat'].values
ax.scatter(lons_s, lats_s, c='#D73027', s=20, marker='^',
zorder=5,

```

```

        edgecolors='#7B0000', linewidths=0.3,
label='USCRN station')
ax.legend(loc='lower right', fontsize=10, framealpha=0.9)
ax.grid(False)
ax.set_title('Fig. 1.3.1 2-Degree Interpolation Grid
over Contiguous USA\n'
            f'Grid: {len(grid_lats)} lat ×
{len(grid_lons)} lon = '
            f'{len(grid_lats)*len(grid_lons)} grid
cells',
            pad=12, fontsize=12)
save(fig, 'method', 'Fig1_3_1_Interpolation_Grid.png')

# Fig 1.3.2 – Variogram model comparison curves
fig, ax = plt.subplots(figsize=(10, 6))
h = np.linspace(0, 1, 500)
nugget, sill, a = 0.05, 1.0, 0.6
gamma_sph = np.where(h <= a,
    nugget + (sill-nugget)*(1.5*(h/a) - 0.5*(h/a)**3),
    sill)
gamma_exp = nugget + (sill-nugget)*(1 - np.exp(-3*h/a))
gamma_gau = nugget + (sill-nugget)*(1 -
np.exp(-3*(h/a)**2))
ax.plot(h, gamma_sph, color='#4DAC26', lw=2.5,
label='Spherical')
ax.plot(h, gamma_exp, color='#D7191C', lw=2.5,
linestyle='--', label='Exponential')
ax.plot(h, gamma_gau, color='#F4A582', lw=2.5,
linestyle=':', label='Gaussian')
ax.axhline(sill, color='gray', lw=1, ls='--',
alpha=0.6)
ax.axhline(nugget, color='gray', lw=1, ls='--',
alpha=0.6)
ax.axvline(a, color='gray', lw=1, ls='--',
alpha=0.6)
ax.text(1.02, sill, 'Sill', va='center', fontsize=10,
color='gray')
ax.text(1.02, nugget, 'Nugget', va='center', fontsize=10,
color='gray')

```

```

ax.text(a, -0.08, 'Range (a)', ha='center', fontsize=10,
color='gray')
ax.set_xlabel('Lag distance (h)', fontsize=11)
ax.set_ylabel('Semivariance  $\gamma(h)$ ', fontsize=11)
ax.legend(fontsize=11, framealpha=0.9)
ax.set_ylim(-0.05, 1.15)
ax.set_title('Fig. 1.3.2 Theoretical Variogram Models
Used in Ordinary Kriging\n'
            'Spherical, Exponential, and Gaussian models
– normalised sill = 1.0',
            pad=12, fontsize=12)
save(fig, 'method', 'Fig1_3_2_Variogram_Models.png')

#
=====

# SECTION 1.4 – PERFORMANCE EVALUATION
#
=====

print('\n[3/5] Section 1.4 – Performance Evaluation')

# Fig 1.4.1 – RMSE grouped bar chart
rmse_vals = {
    'IDW': [3.2023, 4.2132, 11.167,
4.4810, 0.0957],
    'Kriging Spherical': [2.8568, 4.2347, 10.939,
4.4113, 0.0939],
    'Kriging Exponential': [3.2737, 4.5166, 11.017,
4.5096, 0.0967],
    'Kriging Gaussian': [4.6263, 3.7159, 12.880,
5.0081, 0.0978],
}
fig, ax = plt.subplots(figsize=(14, 6))
x = np.arange(5); w = 0.2
for i, (method, vals) in enumerate(rmse_vals.items()):
    bars = ax.bar(x + i*w - 0.3, vals, w, label=method,
color=list(METHOD_COLORS.values())[i],

```

```

        edgecolor='white', linewidth=0.5)
ax.set_xticks(x)
ax.set_xticklabels([f'{l}\n({u})' for l, u in
zip(VAR_LABELS, VAR_UNITS)],
                    fontsize=10)
ax.set_ylabel('RMSE', fontsize=11)
ax.legend(fontsize=10, framealpha=0.9, ncol=2)
ax.set_title('Fig. 1.4.1 RMSE Comparison – All
Interpolation Methods and Variables\n'
            'Lower RMSE indicates better agreement with
ERA5-Land reference data',
            pad=12, fontsize=12)
save(fig, 'performance', 'Fig1_4_1_RMSE_Comparison.png')

# Fig 1.4.2 – Correlation grouped bar chart
corr_vals = {
    'IDW': [0.9624, 0.6478, 0.8318,
0.9296, 0.6607],
    'Kriging Spherical': [0.9699, 0.6293, 0.8405,
0.9379, 0.6835],
    'Kriging Exponential': [0.9602, 0.4417, 0.8391,
0.9319, 0.6502],
    'Kriging Gaussian': [0.9248, 0.6501, 0.7912,
0.8964, 0.6323],
}
fig, ax = plt.subplots(figsize=(14, 6))
for i, (method, vals) in enumerate(corr_vals.items()):
    ax.bar(x + i*w - 0.3, vals, w, label=method,
           color=list(METHOD_COLORS.values())[i],
           edgecolor='white', linewidth=0.5)
ax.axhline(1.0, color='gray', lw=0.8, ls='--', alpha=0.5)
ax.set_ylim(0, 1.08)
ax.set_xticks(x)
ax.set_xticklabels([f'{l}\n({u})' for l, u in
zip(VAR_LABELS, VAR_UNITS)],
                    fontsize=10)
ax.set_ylabel('Pearson Correlation (r)', fontsize=11)
ax.legend(fontsize=10, framealpha=0.9, ncol=2)

```



```

ax.set_title('Fig. 1.4.2 Pearson Correlation Comparison
- All Interpolation Methods\n'
            'Higher r indicates stronger linear
agreement with ERA5-Land reference',
            pad=12, fontsize=12)
save(fig, 'performance',
'Fig1_4_2_Correlation_Comparison.png')

# Fig 1.4.3-1.4.7 - Scatter plots (IDW + Kriging
Spherical vs ERA5)
for vi, var in enumerate(VARIABLES):
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    era5 = get_era5(var).ravel()
    for ai, (method_name, results) in enumerate([
        ('IDW', results_idw),
        ('Kriging Spherical',
results_kriging_all['spherical']),
    ]):
        ax = axes[ai]
        pred = results[var].ravel()
        mask = ~(np.isnan(pred) | np.isnan(era5))
        p_c, e_c = pred[mask], era5[mask]
        # Sample 50k for plotting speed
        if len(p_c) > 50000:
            idx = np.random.choice(len(p_c), 50000,
replace=False)
            p_c, e_c = p_c[idx], e_c[idx]
        r, _ = pearsonr(p_c, e_c)
        rmse = np.sqrt(np.mean((p_c - e_c)**2))
        ax.scatter(e_c, p_c, s=1, alpha=0.15,
color=list(METHOD_COLORS.values())[ai],
            rasterized=True)
        lims = [min(e_c.min(), p_c.min()), max(e_c.max(),
p_c.max())]
        ax.plot(lims, lims, 'k--', lw=1, label='1:1
line')
        ax.set_xlabel(f'ERA5 ({VAR_UNITS[vi]})',
            fontsize=11)

```

```

        ax.set_ylabel(f'{method_name} ({VAR_UNITS[vi]})',
fontSize=11)
        ax.set_title(f'{method_name}', fontsize=12,
fontWeight='bold')
        ax.text(0.05, 0.92, f'r = {r:.4f}\nRMSE =
{rmse:.4f} {VAR_UNITS[vi]}',
                transform=ax.transAxes, fontsize=10,
                bbox=dict(boxstyle='round', fc='white',
alpha=0.8))
        ax.legend(fontsize=9)
        fig.suptitle(f'Fig. 1.4.{vi+3} Scatter Plot –
{VAR_LABELS[vi]} '
                    f'vs ERA5-Land Reference\nIDW (left) and
Kriging Spherical (right) '
                    f'– full 2006-2021 period (n ≈ 2.2M
grid-day values)',
                    fontsize=12, y=1.01)
        plt.tight_layout()
        save(fig, 'performance',

f'Fig1_4_{vi+3}_Scatter_{var.title().replace("_", "")}.png')

#

# SECTION 1.5 – STATISTICAL ANALYSIS
#

print('\n[4/5] Section 1.5 – Statistical Analysis')

# Fig 1.5.1 – Box plots (IDW vs ERA5) for all variables
fig, axes = plt.subplots(1, 5, figsize=(20, 7))
for vi, var in enumerate(VARIABLES):
    ax = axes[vi]
    era5 = get_era5(var).ravel()
    era5 = era5[~np.isnan(era5)]

```

```

    idw = results_idw[var].ravel()
    idw = idw[~np.isnan(idw)]
    if len(era5) > 100000:
        era5 = np.random.choice(era5, 100000,
replace=False)
        idw = np.random.choice(idw, 100000,
replace=False)
    bp = ax.boxplot([idw, era5],
                    labels=['IDW', 'ERA5'],
                    patch_artist=True,
                    medianprops=dict(color='black',
lw=2),
                    flierprops=dict(marker='.', ms=1,
alpha=0.2),
                    whiskerprops=dict(lw=1.2),
                    capprops=dict(lw=1.2))
    bp['boxes'][0].set_facecolor(METHOD_COLORS['IDW'] +
'88')
    bp['boxes'][1].set_facecolor(METHOD_COLORS['ERA5'] +
'88')
    ax.set_title(f'{VAR_LABELS[vi]}\n({VAR_UNITS[vi]})',
fontsize=10, fontweight='bold')
    ax.set_xlabel('')
fig.suptitle('Fig. 1.5.1 Box Plot Comparison – IDW
Interpolated vs ERA5-Land\n'
            'Showing median, interquartile range, and
outliers (full 2006–2021 dataset)',
            fontsize=12, y=1.01)
plt.tight_layout()
save(fig, 'stats', 'Fig1_5_1_Boxplots_IDW_vs_ERA5.png')

# Fig 1.5.2 – CDF comparison (KS test visualisation) for
all variables
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.ravel()
for vi, var in enumerate(VARIABLES):
    ax = axes[vi]
    era5 = get_era5(var).ravel(); era5 =
era5[~np.isnan(era5)]

```

```

    idw = results_idw[var].ravel(); idw =
idw[~np.isnan(idw)]
    if len(era5) > 50000:
        era5 = np.random.choice(era5, 50000,
replace=False)
        idw = np.random.choice(idw, 50000,
replace=False)
    era5_s = np.sort(era5); idw_s = np.sort(idw)
    ax.plot(era5_s, np.linspace(0, 1, len(era5_s)),
            color=METHOD_COLORS['ERA5'], lw=2,
label='ERA5')
    ax.plot(idw_s, np.linspace(0, 1, len(idw_s)),
            color=METHOD_COLORS['IDW'], lw=2, ls='--',
label='IDW')
    ax.set_xlabel(VAR_UNITS[vi], fontsize=10)
    ax.set_ylabel('Cumulative probability', fontsize=10)
    ax.set_title(f'{VAR_LABELS[vi]}', fontsize=11,
fontweight='bold')
    ax.legend(fontsize=9); ax.grid(True, alpha=0.3)
axes[5].set_visible(False)
fig.suptitle('Fig. 1.5.2 Empirical CDF Comparison – IDW
vs ERA5-Land\n'
            'Basis for Kolmogorov-Smirnov two-sample
test; D = max vertical distance',
            fontsize=12, y=1.01)
plt.tight_layout()
save(fig, 'stats',
'Fig1_5_2_CDF_Comparison_IDW_ERA5.png')

# Fig 1.5.3 – KS D-statistic heatmap
ks_matrix = np.array([
    [0.0300, 0.2184, 0.0506, 0.1020, 0.1960],
    [0.0334, 0.2320, 0.0667, 0.1150, 0.2604],
    [0.0355, 0.3184, 0.0642, 0.1092, 0.2518],
    [0.0809, 0.4413, 0.1384, 0.0787, 0.3059],
])
fig, ax = plt.subplots(figsize=(12, 5))
im = ax.imshow(ks_matrix, cmap='RdYlGn_r', aspect='auto',
vmin=0, vmax=0.5)

```

```

plt.colorbar(im, ax=ax, label='KS D-statistic (0 =
identical, 0.5 = large difference)')
ax.set_xticks(range(5))
ax.set_xticklabels([f'{l}\n({u})' for l, u in
zip(VAR_LABELS, VAR_UNITS)], fontsize=10)
ax.set_yticks(range(4))
ax.set_yticklabels(['IDW', 'Kriging Spherical',
                    'Kriging Exponential', 'Kriging
Gaussian'], fontsize=10)
for i in range(4):
    for j in range(5):
        txt = f'{ks_matrix[i,j]:.4f}'
        col = 'white' if ks_matrix[i,j] > 0.3 else
'black'
        ax.text(j, i, txt, ha='center', va='center',
fontsize=10,
                color=col, fontweight='bold')
ax.set_title('Fig. 1.5.3 KS Test D-Statistic Heatmap –
All Methods vs ERA5-Land\n'
            'Green = similar distribution, Red = large
distributional difference',
            pad=12, fontsize=12)
plt.tight_layout()
save(fig, 'stats', 'Fig1_5_3_KS_Dstatistic_Heatmap.png')

#
=====

# SECTION 1.6 – SEASONALITY
#
=====

print('\n[5/5] Section 1.6 – Seasonality')

# Fig 1.6.1-1.6.5 – Seasonal mean bar charts (all methods
+ ERA5)
for vi, var in enumerate(VARIABLES):
    era5_arr = get_era5(var)

```

```

fig, axes = plt.subplots(1, 4, figsize=(20, 5),
sharey=False)
for si, season in enumerate(SEASON_LABELS):
    ax = axes[si]
    s_idx = [i for i, s in enumerate(seasons_list) if
s == season]
    means = []
    labels = []
    colors = []
    for method_name, res in [
        ('IDW', results_idw),
        ('Kriging\nSpherical',
results_kriging_all['spherical']),
        ('Kriging\nExponent.',
results_kriging_all['exponential']),
        ('Kriging\nGaussian',
results_kriging_all['gaussian']),
        ('ERA5', None),
    ]:
        if method_name == 'ERA5':
            arr = era5_arr[s_idx].ravel()
        else:
            arr = res[var][s_idx].ravel()
        arr = arr[~np.isnan(arr)]
        means.append(np.mean(arr))
        labels.append(method_name)
        key = method_name.replace('\n', '
').replace('Exponent.', 'Exponential')
        colors.append(METHOD_COLORS.get(key,
'#888888'))
    bars = ax.bar(range(5), means, color=colors,
edgecolor='white', linewidth=0.5,
width=0.7)
    for bar, val in zip(bars, means):
        ax.text(bar.get_x() + bar.get_width()/2,
bar.get_height() +
abs(bar.get_height())*0.01,
f'{val:.2f}', ha='center',
va='bottom',

```

```

        fontsize=7.5, fontweight='bold')
    ax.set_xticks(range(5))
    ax.set_xticklabels(labels, fontsize=8.5)
    ax.set_title(season, fontsize=11,
fontweight='bold',
                color=SEASON_COLORS[si])
    if si == 0:
        ax.set_ylabel(f'Mean {VAR_LABELS[vi]}
({VAR_UNITS[vi]})', fontsize=10)
    fig.suptitle(f'Fig. 1.6.{vi+1} Seasonal Mean -
{VAR_LABELS[vi]}\n'
                f'All interpolation methods vs ERA5-Land
reference - 2006-2021',
                fontsize=12, y=1.01)
    plt.tight_layout()
    save(fig, 'seasonal',

    f'Fig1_6_{vi+1}_SeasonalMean_{var.title().replace("_", ""
)}.png')

# Fig 1.6.6-1.6.10 – Seasonal spatial maps (IDW, 4
seasons)
season_dates = {
    'DJF (Winter)': '2010-01-15',
    'MAM (Spring)': '2010-04-15',
    'JJA (Summer)': '2010-07-15',
    'SON (Autumn)': '2010-10-15',
}
for vi, var in enumerate(VARIABLES):
    fig, axes = plt.subplots(1, 4, figsize=(22, 5))
    idw_maps = []
    for si, (season, date_str) in
enumerate(season_dates.items()):
        t_idx =
list(date_range_local).index(pd.Timestamp(date_str))
        idw_maps.append(results_idw[var][t_idx])
        vmin = np.nanpercentile(np.concatenate([m.ravel() for
m in idw_maps]), 2)

```

```

    vmax = np.nanpercentile(np.concatenate([m.ravel() for
m in idw_maps]), 98)
    for si, (season, date_str) in
enumerate(season_dates.items()):
        ax = axes[si]
        pcm = ax.pcolormesh(grid_lons, grid_lats,
idw_maps[si],
                                cmap=VAR_CMAPS[vi],
shading='auto',
                                vmin=vmin, vmax=vmax)
        plt.colorbar(pcm, ax=ax, label=VAR_UNITS[vi],
shrink=0.85, pad=0.02)
        ax.set_title(season, fontsize=11,
fontweight='bold',
                                color=SEASON_COLORS[si])
        ax.set_xlabel('Longitude', fontsize=9)
        ax.set_ylabel('Latitude',  fontsize=9)
        ax.set_xlim([-130, -60]); ax.set_ylim([22, 52])
        ax.grid(True, ls='--', alpha=0.3)
        ax.text(0.02, 0.96, date_str,
transform=ax.transAxes,
                                fontsize=8, va='top', style='italic')
        fig.suptitle(f'Fig. 1.6.{vi+6} Seasonal Spatial Maps
- {VAR_LABELS[vi]} (IDW)\n'
                    f'Representative days per season -
shared colour scale '
                    f'({vmin:.2f}-{vmax:.2f}
{VAR_UNITS[vi]})',
                                fontsize=12, y=1.01)
        plt.tight_layout()
        save(fig, 'seasonal',

        f'Fig1_6_{vi+6}_SeasonalMaps_IDW_{var.title().replace("_",
"")}.png')

# Fig 1.6.11-1.6.15 - Method comparison maps (one
representative summer day)
target_date = pd.Timestamp('2010-07-15')
t_idx = list(date_range_local).index(target_date)

```



```

for vi, var in enumerate(VARIABLES):
    fig, axes = plt.subplots(1, 4, figsize=(22, 5))
    maps = [
        results_idw[var][t_idx],
        results_kriging_all['spherical'][var][t_idx],
        results_kriging_all['exponential'][var][t_idx],
        results_kriging_all['gaussian'][var][t_idx],
    ]
    names = ['IDW', 'Kriging Spherical', 'Kriging
Exponential', 'Kriging Gaussian']
    vmin = np.nanpercentile(np.concatenate([m.ravel()
for m in maps]), 2)
    vmax = np.nanpercentile(np.concatenate([m.ravel()
for m in maps]), 98)
    for ai, (m, name) in enumerate(zip(maps, names)):
        ax = axes[ai]
        pcm = ax.pcolormesh(grid_lons, grid_lats, m,
                           cmap=VAR_CMAPS[vi],
shading='auto',
                           vmin=vmin, vmax=vmax)
        plt.colorbar(pcm, ax=ax, label=VAR_UNITS[vi],
shrink=0.85, pad=0.02)
        ax.set_title(name, fontsize=10,
fontweight='bold')
        ax.set_xlabel('Longitude', fontsize=9)
        ax.set_ylabel('Latitude', fontsize=9)
        ax.set_xlim([-130, -60]); ax.set_ylim([22, 52])
        ax.grid(True, ls='--', alpha=0.3)
        fig.suptitle(f'Fig. 1.6.{vi+1} Method Comparison
Spatial Maps - '
                    f'{VAR_LABELS[vi]}
({target_date.date()})\n'
                    f'All four interpolation methods -
shared colour scale '
                    f'({vmin:.2f}-{vmax:.2f}
{VAR_UNITS[vi]})',
                    fontsize=12, y=1.01)
    plt.tight_layout()

```

```

        save(fig, 'seasonal',

f'Fig1_6_{vi+11}_MethodComparison_{var.title().replace("
_", "")}.png')

print('\n' + '=' * 60)
print('ALL FIGURES SAVED!')
print(f'Location: {BASE}/')
print('=' * 60)
print('\nSubfolders:')
for k, v in FOLDERS.items():
    files = os.listdir(v)
    print(f'    {v.split("/")[-1]}/    ({len(files)}
figures)')

```